



中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

学 校

南开大学

参赛队号

25100550012

队员姓名

- | | |
|----|-----|
| 1. | 姜政旭 |
| 2. | 于雪婧 |
| 3. | 徐志骏 |
-

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目： 面向 NPU 的缓存感知调度与流水优化方法研究

摘 要：

在神经网络推理过程中，基于单指令多数据流（SIMD）架构的通用神经网络加速处理器（NPU）硬件适合处理边缘推理任务，但其软件模型适配上存在问题。通用调度算法可以自动给出算子的最优调度，实现内存与计算协同管理，成为处理器大规模推广的关键。

基于问题一，本文给出了节点调度算法。首先基于问题的 NP-hard 性值，本文使用启发式算法中的贪心算法，针对特殊缓存 L0，分析 L0 级缓冲区节点控制关系并设计在计算图中加边以等价控制关系的策略，实现了对 L0 约束的满足，最终确定了节点调度算法并应用于示例计算图，缓存容量最高提升 47%，时间复杂度控制在 $O(N^2 + NM)$ 。

问题二在问题一的基础上进行缓存分配。基于初步拓扑序，本文首先使用 Best-fit 算法对节点分配缓存，使用空闲列表来维护空闲地址，在分配中出现的缓存死锁情况，则进行 SPILL 操作。SPILL_OUT 节点的目标缓存区的选取受到缓冲区 size、下一操作位置以及 COPY_OUT 节点的使用情况的影响，本文设计 WCB 评分模型打分，并通过滑动寻找最小 WCB 的被占用缓冲区集合（MATCH 算法）来确定 SPILL_OUT 节点的目标缓存区，SPILL_IN 节点的数据搬入时，沿用 Best-fit 算法分配缓存。综上，在问题一二的算法构成了完整的调度算法。首先通过节点调度算法生成计算图的初始拓扑序列，基于该序列采用 Best-Fit 算法完成初始缓存空间分配；当检测到缓存死锁导致分配失败时，触发 SPILL 操作进行缓存空间重整，在 SPILL 操作引发计算图状态更新后，再次调用节点调度算法对后续节点的拓扑序列进行动态调整，形成“调度-分配-死锁处理-动态更新”的闭环优化机制，保障缓存分配的有效性与计算图执行的连贯性。详细 SPILL 计量和搬运量结果见表7.6。

对于问题三，除了数据额外搬运量，还需要将时钟数纳入优化考虑因素。我们引入了 ASAP 方法（见公式8.3.2），题目中允许依据执行单元进行操作节点的流水排布是 ASAP 的根基，操作节点若满足三个前置条件：无执行依赖、执行单元空闲、物理地址已分配，即可提前在执行单元并行计算以减少等待时钟。最终在劣化数据搬运量情况下，平均可以降低大约 20%-40% 的总时钟（结果见表8.8）。

关键词： 启发式算法 节点调度 动态缓存分配 Best-Fit 算法

目录

1	问题背景	4
2	问题重述	4
2.1	问题一：最小缓存驻留制度	5
2.2	问题二：缓存分配与换入换出	6
2.3	问题三：性能优化策略	6
3	问题分析	6
3.1	对问题一的分析：最小缓存驻留制度	6
3.2	对问题二的分析：缓存分配与换入换出	7
3.3	对问题三的分析：混合指标下性能优化策略	8
4	模型假设	8
5	符号说明	9
6	问题一的建立与求解	9
6.1	贪心算法	9
6.1.1	问题分解：	9
6.1.2	贪心准则：	10
6.2	节点调度算法构建	12
6.3	算法时间复杂度分析	15
7	问题二的建立与求解	15
7.1	缓存初始分配	15
7.2	SPILL 操作	16
7.2.1	SPILL_OUT 数据的搬出	16
7.2.2	SPILL_IN 数据的搬入	17
7.2.3	SPILL 操作节点的属性更新	17
7.2.4	SPILL 操作节点的计算图更新	17
7.2.5	总额外数据搬运量	18
7.3	缓存冲突管理方案	18
7.3.1	WCB 评分模型	19

7.3.2	SPILL_OUT 操作的目标缓冲区选取——MATCH 算法	19
7.4	调度算法综述	21
7.5	算法应用结果	23
8	问题三的建立与求解	23
8.1	流水排布	23
8.2	计算图相关时钟设定	24
8.3	基于空闲压缩的 ASAP 方法	26
8.3.1	ASAP 方法的有效性	26
8.3.2	实现原理	26
8.4	求解结果	27
9	灵敏度分析	27
10	模型评价	29
10.1	模型优缺点	29
10.1.1	模型优点	29
10.1.2	模型不足	29
10.2	模型改进	29
	参考文献	31
	附录 A 支撑材料文件	33
1.1	Attachment.rar	33
	附录 B Python 源程序	34
B.1	第 1 问程序	34
B.2	第 2 问第 3 问程序	44
B.3	灵敏度分析程序	54

1 问题背景

在神经网络推理过程中，基于单指令多数据流（SIMD）架构的通用神经网络加速处理器（NPU）硬件设计简单，面效高，适合于处理对功耗、面积、成本具有一定要求的边缘推理任务。但由于神经网络的软件模型充满“异构性、动态性、不规则性”，所以该处理器在软件模型适配上存在挑战。

目前，针对软件适配性的解决方案是人工拆解和编排算子的计算过程，但这种方法的难度大且通用性低，成为上述处理器大规模推广的瓶颈。因此，通用调度算法显得尤为重要。算法不仅要能够自动编排计算图上的原子最优操作调度，更需考虑 SIMD 平台的硬件限制和时间，实现内存与计算协同动态管理。

具体地，核内调度算法需要在给定计算图的基础上，提供包括操作节点和缓存管理节点在内的调度顺序以及缓存管理节点的缓存分配规则，必要时使用缓存换入换出操作（SPILL 操作）进行数据搬运以暂时释放硬盘缓存空间。

2 问题重述

首先根据题意，本文首先对调度算法的目标进行明确：

1. 节点调度顺序：计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ 为有向无环图（DAG），其节点 v_i 分为两类：特定硬件单元上执行的操作节点和用于缓存资源分配的缓存管理节点，操作节点和缓存管理节点的关键属性如表 2.1。

表 2.1 节点属性说明

节点类型	属性	说明
操作节点	Id	从 0 开始的节点唯一标识符
	Op	操作指令名，可为除 ALLOC/FREE 以外的任意操作名
	Pipe	指定该操作在哪个执行单元上运行（如 Cube、Vector、FIXP 等）
	Cycles	表示在该执行单元上操作运行所需的时钟周期数
	Bufs	表示该操作所需输入和输出数据缓冲区的唯一标识符（BufId）组成的列表
缓存管理节点	Id	从 0 开始的节点唯一标识符
	Op	ALLOC 或 FREE，分别代表申请或释放数据缓冲区
	BufId	所申请或释放的数据缓冲区的唯一标识符
	Size	申请或释放的数据缓冲区长度
	Type	该数据缓冲区所处的缓存类型（如 L1/UB/L0A/L0B/L0C 等）

计算图中的每条有向边表示节点之间的执行依赖关系，节点必须严格依赖关系依次执行，故计算图所有节点在内的一个拓扑序，即节点调度顺序，是调度算法的首要目标。

2. 缓存地址分配：硬件上的各类缓存（如 L1/UB/L0A/L0B/L0C 等）均为有限长度的连续地址空间，**数据缓冲区在缓存中占据的是一片连续地址空间**。由于调度算法中涉及到数据缓冲区的申请和释放（ALLOC 和 FREE），所以调度算法还需要对于缓冲区进行管理。具体地，申请或释放的数据缓冲区在缓存中占据连续地址空间的大小由 Size 字段制定，位置由 Offset 和 Size 共同确定。

在给定的调度序列上，一个数据缓冲区的生命周期为其 ALLOC 节点所在位置至其 FREE 节点所在位置，生命周期不重叠的数据缓冲区可以先后复用同一片缓存区域，但生命周期重叠的数据缓冲区需分配不重叠的缓存区域。

3. 缓存换入换出 SPILL 操作：由于硬件缓存的限制，可能会出现在执行的某个时刻硬件空间无法满足所有缓冲区需要的情况。此时，引入缓存换入换出 SPILL 操作，将部分缓冲区数据临时搬至核外空间 DDR 中，计算需要时重新载入。

具体地，若引入一次 SPILL 操作，则插入两个新增操作节点 SPILL_IN 和 SPILL_OUT（相应增加依赖边），表示将硬件上的缓存数据搬出和搬入。对于 SPILL_OUT 节点需关注被换出缓冲区的选择，对于 SPILL_IN 节点，搬入数据可加载至新的核内缓存空间并分配缓存。

随后，本文关注调度算法的评估指标。

- **任务总执行时间：**表示硬件上执行完计算图需要的总时钟周期数，主要受节点的依赖边关系以及缓存物理地址复用带来的执行依赖影响；
- **总额外数据搬运量：**SPILL 操作引起的额外 DDR 访问量，若目标缓冲区在计算图中被 COPY_IN 节点使用，则额外数据搬运量是 $2 \times \text{Size}$ ，否则为 Size。

2.1 问题一：最小缓存驻留制度

问题一规定了对于某个计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ ，每个节点 v 需要的缓存大小：

$$\mathcal{M}(v) = \begin{cases} \text{Size}(v) & \text{if Op}(v) = \text{ALLOC} \\ -\text{Size}(v) & \text{if Op}(v) = \text{FREE} \\ 0 & \text{otherwise} \end{cases}$$

相应的最大缓存容量 $\max(V_{\text{stay}})$ 的计算规则如下：令初始驻留缓存容量 $V_{\text{stay}} = 0$ ，按节点调度顺序 S 遍历各节点；若遇到 ALLOC 节点，则 V_{stay} 增加相应大小，若遇到 FREE 节点，则 V_{stay} 减小相应大小。在此过程中， V_{stay} 的最大值即为需要驻留的最大缓存容量 $\max(V_{\text{stay}})$ 。其中，由于缓存容量仅记录 UB 和 L1，且要求单个 L0 级缓存在任意时刻最多

只能分配一个缓冲区，若 ALLOC 节点 v 申请 L0 级缓存，则可认为在问题一中 $\text{Size}(v)=0$ 。

问题一要求设计一种节点调度算法，使得最大缓存容量 $\max(V_{\text{stay}})$ 尽可能小，并分析算法相对于节点总数 N 的时间复杂度，同时对六个示例计算图应用该调度算法得出结果。

2.2 问题二：缓存分配与换入换出

基于问题一得到的节点调度算法，问题二将**缓存分配策略**纳入讨论范围。

问题二给出了 L1、UB、L0A、L0B、L0C 的具体缓存容量限制，如表 2.2。

表 2.2 变量符号及其说明

缓存	容量
L1	4096
UB	1024
L0A	256
L0B	256
L0C	512

要求基于问题一中得到的调度序列，设计缓存分配方案，使得额外数据搬运量尽可能小。(必要时可对问题一中的调度序列进行优化调整)。题目要求给出每个缓冲区对应的地址偏移、SPILL 操作列表、总额外数据搬运量和完整调度序列。

2.3 问题三：性能优化策略

基于以上两个问题，已得到包括调度顺序和缓存分配方案在内的完整调度算法，需要针对算法评估指标中的**总运行时间指标**进行进一步优化。

问题三要求通过优化调度顺序或缓存分配方案，针对总额外数据搬运量和算子总运行时间同步优化，并给出两项指标的提升效果。

3 问题分析

3.1 对问题一的分析：最小缓存驻留制度

由于问题一要求给出一种节点调度算法，使得调度最大缓存容量尽可能小。由于实际问题以及本题所涉及到的计算图均非简单的计算图，其包含节点较多，较为复杂，节点顺序组合容易造成解空间爆炸，同时复杂的约束也会造成计算复杂度的显著上升，因为，本文采用**启发式算法**中的**贪心算法**来获得尽可能小的最大缓存容量调度顺序。

贪心算法 (Greedy Algorithm) 是一种通过每一步都做出当前局部最优选择来寻求全局最优解的启发式策略。它的核心思想是不考虑未来步骤的影响, 仅依据当前信息做出决策, 最终期望通过一系列局部最优选择累积得到全局最优解。

首先, 本文将原问题分解为一系列可逐步求解的子问题。在本题中, 考虑在某个节点执行后: 会出现若干个前序节点都执行完且其他执行条件均满足的节点, 即当前可执行的节点, 本文将其定义为该时间点上的候选节点集, 此时节点调度问题转化为在该时间点的候选节点集中选取优先执行节点的问题。

随后, 本文定义贪心准则。贪心准则是在每个子问题中决策中, 所要遵循的原则。根据题意, 在尽可能减小最大缓存容量的目标下, 在优先执行节点选择上, 需遵循以下原则:

- 对于已经可以释放的数据缓冲区, 应尽快释放以避免缓冲区占用。
- 对于 Type=L0A、L0B 和 L0C 的 ALLOC 缓存管理节点, 当且仅当前序执行过程未申请对应空间才允许执行。
- 因 L0 级缓冲区遵循申请后尽快释放原则, 所以应尽快对涉及 L0 级缓冲区的操作节点进行执行, 以便尽快释放 L0 级缓冲区。

进一步, 本文基于已经确定的贪心准则进行迭代选择。在每个节点执行完之后, 更新当前候选节点集, 并依据贪心准则判断下一个要执行的节点, 直至图中所有节点均被执行完毕。在此过程中记录被执行的节点顺序即所求拓扑序。

最后, 本文进行可行性分析。基于以上步骤, 本文验证题中图 1 生成节点拓扑序列, 与题中给出的有效拓扑序列一致, 并搜集多个实际计算图进一步验证上述算法可行。

3.2 对问题二的分析: 缓存分配与换入换出

基于问题一得到的节点调度算法, 问题二将**缓存分配策略**纳入讨论范围。根据题意, 缓存的分配可以分为两部分执行:

缓存初步分配: 首先基于题中给出的缓存容量, 本文选取 Best-fit 算法进行初步分配。为减少缓存碎片化, 对于缓存的初步分配时, 选择最小的能够容纳的连续缓存空间, 避免缓存空间的浪费。

SPILL 操作: 在初步分配过程中, 如果遇到缓存死锁现象, 即缓存空间中剩余空间不足以满足新指令申请的需要, 则进行 SPILL 操作。

SPILL 操作增加 SPILL_OUT 和 SPILL_IN 两个新的操作节点, 在题目已经对新操作节点的属性和图的更新规则定义的情况下, 本文需考虑 SPILL_OUT 节点所搬出的数据选择问题, 以及 SPILL_IN 节点将数据搬回的时间以及搬回的新地址。

SPILL_OUT 的搬出对象需满足新空间申请需求, 本文通过 WCB 打分和 MATCH 算法来完成; 由于 SPILL_IN 可将数据加载至新的核内缓存地址区间, 所以在当被移出数据需

要使用时，通过 `SPILL_IN` 操作在结果上可以视为在初步分配时的 `ALLOC` 操作，故本文仍按照初步分配的 `Best-fit` 算法进行地址分配。

由于 `SPILL` 操作加入了新的操作节点导致计算图的更新，对于 `SPILL` 操作后面执行的节点，本文使用更新后的计算图运用问题一中所得的节点调度算法进行拓扑序的更新调整。

3.3 对问题三的分析：混合指标下性能优化策略

问题三要求同时优化数据额外搬运量时钟数这两个指标，在前一问的基础上，允许流水排布优化执行，即允许多个执行单元展开并行计算。因此尝试引入方法设计合理的流水排布方案，即探讨在什么条件下，基于不同执行单元的操作节点可以与其他执行单元中的操作节点存在时钟意义下重叠，如果可能的话将所有满足重叠条件的操作节点提前到最早可以开始执行的时钟位置，以此降低总任务执行时间，同时也不会劣化额外搬运量这一指标。

4 模型假设

1. 计算图均为有向无环图。
2. 硬件上的各类**缓存**（如 `L1/UB/L0A/L0B/L0C` 等）均为有限长度的连续地址空间，**数据缓冲区**在缓存中占据的是一片连续地址空间，其行为完全由程序控制。
3. 生命周期不重叠的数据缓冲区可以先后复用同一片缓存区域，但生命周期重叠的数据缓冲区需分配不重叠的缓存区域。
4. 问题一：仅关注调度顺序；仅考虑 `L1` 和 `UB` 类型的缓存使用量；不考虑实际硬件上 `L1` 和 `UB` 缓存的长度限制；针对 `L0` 级缓冲区（`L0A`、`L0B`、`L0C`），遵循申请后尽快释放原则，要求 `L0A`、`L0B`、`L0C` 分别同时最多只能有一个缓冲区驻留。
5. `SIMD` 架构的处理核心为多个可并行的执行单元和多级缓存，系统能够基于调度序列自动插入各单元同步指令，由调度中心分发指令流。
6. `SPILL_IN` 节点可将数据加载至新的核内缓存地址空间。
7. 忽略缓存管理节点，所有的计算图都从 `COPY_IN` 节点开始，以 `COPY_OUT` 节点结束。
8. 在 7.3 中，假设问题一生成的原始拓扑序列和加入 `SPILL_IN` 和 `SPILL_OUT` 节点生成的新拓扑序列差异在可接受的范围内。

5 符号说明

本文主要用到的数学符号列举在表 5.3 中。

表 5.3 变量符号及其说明

符号	符号说明
$\mathcal{G} = (V, E)$	计算图
v_j	节点，各属性定义如表表 2.1
N 、 M	计算图中的节点总数和依赖边数量
D_j 、 H	表示 $Id = j$ 的节点入度和候选节点集。
Tag	三维数组，分别记录 Type 为 L0A、L0B、L0C 的缓冲区驻留情况。
V_{stay}	最大缓存容量
DDR	核外内存
$\mathcal{L}_i$	i 类型的缓存空闲列表
$B = (b_1, \dots, b_k)$	被占用缓冲区集合
$F(b_j)$	被占用缓冲区 b_j 的 WCB 分数
w_1 、 w_2	WCB 参数
$c(b_j)$ 、 $size(b_j)$	被占用缓冲区 b_j 的数据搬运量指标、size
$next_uses(b_j)$	被占用缓冲区 b_j 上下一个执行节点在问题一拓扑序中与当前执行节点的距离
offset	缓冲区起始地址偏移
$S(v_i)$ 、 $E(v_i)$	v_i 的起始结束时间

6 问题一的建立与求解

6.1 贪心算法

6.1.1 问题分解：

本题调度问题计算图复杂，节点顺序组合容易造成解空间的爆炸问题；复杂的约束条件使得计算复杂度显著上升，因此属于 NP-hard 问题（已有严格的证明）。本文尝试使用启发式算法在“求解效率”与“解的质量”间寻找平衡。贪心算法作为典型的启发式算法，虽无法保证找到全局最优解，但可以通过**局部最优接近全局最优**。

本文首先将全局节点调度分解为一系列连续的局部决策子问题。在计算图的调度过程中，当某个节点执行完成后，会产生一批满足“所有前序依赖节点已执行完毕”且其他执行条件均满足的节点，这次节点可以立即执行，本文将这些节点定义为当前的**候选节点**，

构成的集合为候选节点集 H 。例如，若节点 A 的前序节点为 B 和 C ，当 B 、 C 均执行完成后， A 为候选节点，加入候选节点集。

具体地，本文对每个节点 v_j 定义入度 D_j ，表示该节点当前被其他节点指向的边的数量，即该节点的所依赖的前序节点。当某个节点入度为 0 时，表示该节点无前序依赖节点，即该节点为候选接节点，加入候选节点集 H 。

因此，全局调度目标被转化为“在每个时间点的候选节点集 H 中，选择节点优先执行”的局部子问题。每完成一次选择并执行节点后，将该节点的后继节点入度减一，以更新计算图的依赖状态，进而更新候选节点集。

通过这种分解，原本复杂的全局调度问题被拆解为一系列独立的候选节点选择步骤，每个步骤的决策范围仅局限于当前的候选节点集，大幅降低了单步决策的复杂度。

6.1.2 贪心准则：

将全局节点调度拆解为局部决策子问题后，子问题的决策依据贪心准则，即如何判断哪个候选节点的执行能带来较小的当前缓存占用，并为后续调度预留更优条件。为更好地确定贪心准则，本文进行如下分析：

a. 初始优先级指标

本文首先设定 $P(v_j) = \mathcal{M}(v_j)$ 为节点 v_j 的初始优先级指标。

在非 L0 缓存中，结合“最小化最大缓存容量”的核心目标，在候选节点集 H 中，选取 $P(v_j)$ 最小的节点作为最高优先级执行。

b. L0 级缓冲区节点依赖

考虑到题目中对 L0 级缓冲区存在申请后尽快释放的要求，并限制 L0A、L0B、L0C 分别同时最多只能有一个缓冲区驻留，所以对涉及到 L0 级缓冲区的节点，应进一步讨论。

对于同一缓存类型的 L0 级缓冲区，题目只允许驻留一个缓冲区。因此，在计算图调度过程中，会出现因地址复用而产生的同缓存类型的 L0 级缓冲区依赖。

如图 6.1，为清晰展示，下图省略了部分 FREE 节点。本文已验证下图所示情形在示例计算图中存在。

在图中所示情况中，初始存在 v_0 、 v_1 、 v_2 三个入度为 0 的节点，且 $\mathcal{M}(v_0) = \mathcal{M}(v_1) = \mathcal{M}(v_2) = 0$ ，（基于问题一假设，不考虑 L0 类型的缓存使用量）此时若首先选择执行 v_0 ，申请 L0A 的缓冲区，且此缓冲区需在 v_7 执行后才可释放， v_7 的执行以 v_2 的执行为先决条件，但在 v_0 执行后已经申请 L0A 的缓冲区，因此此时 v_2 不能在申请 L0A 缓冲区，此时存在 L0 级缓冲区的依赖，计算图无法执行。

因此，当候选节点集中同时出现多个同类型 L0 级缓冲区时，需要考虑 L0 级缓冲区间依赖。

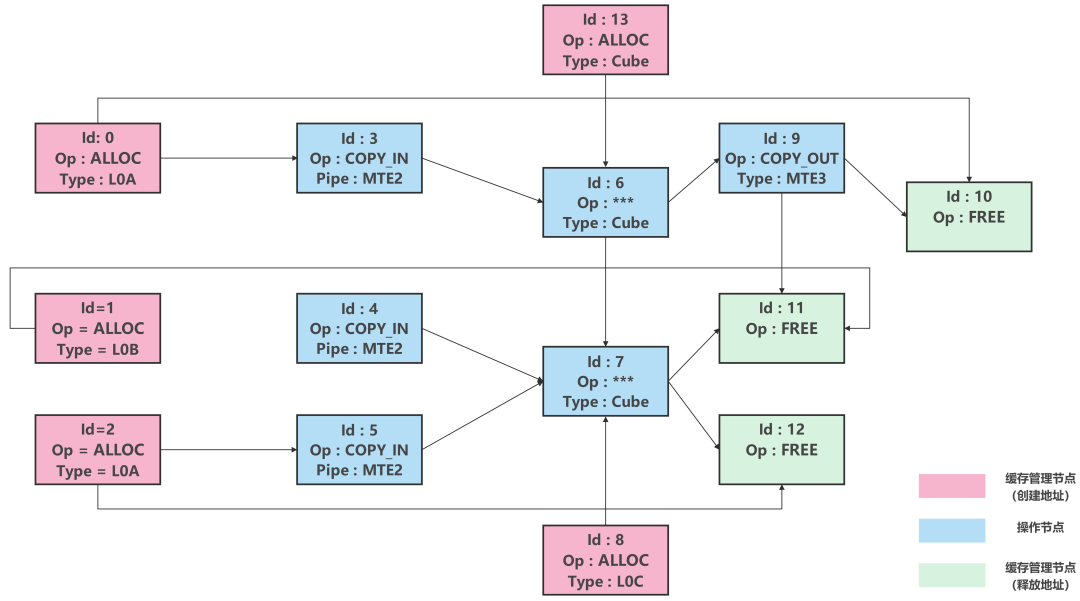


图 6.1 L0 级缓冲区间依赖示意图

L0 级缓冲区 ALLOC 申请依赖处理——计算图加边

首先，在读入计算图的边时，若存在一条边的前后节点分别为 ALLOC 和 FREE 节点，则判断该边连接的 ALLOC 和 FREE 是对应关系。由此，不难得出所有 ALLOC 节点和 FREE 节点的一一对应关系。

随后，本文使用多源 BFS 遍历图上所有节点，不难得到每个 ALLOC 节点到所有 FREE 节点的距离。由此，若某个 L0 级 ALLOC 节点 v_i 到某个 L0 级 FREE 节点 v_{jj} 的距离 >1 ，则说明该 v_i 与该 FREE 节点 v_{jj} 一一对应的 ALLOC 节点 v_j 存在着 L0 级缓冲区依赖关系，且 v_i 控制 v_j 。（此处的“控制”指的是 v_j 的执行必须依赖 v_i 的执行）

Marchal[3] 等提出通过添加依赖边来处理依赖关系的方法，本文将其转化为计算图中的依赖边。即若 ALLOC 节点 v_i 控制 v_j ，且二者为同类型的 L0 级缓存（例如，都为 L0A），则在原计算图中增加从 v_i 指向 v_j 的依赖边。

L0 级缓冲区驻留情况维护

由于本文题中要求限制同类型 L0 级缓冲区的驻留情况，因此本文建立三维数组 Tag 对 L0 级缓冲区的驻留情况进行维护。Tag[0]、Tag[1] 和 Tag[2] 分别表示当前是否存在 L0A、L0B 和 L0C 的缓冲区驻留，0 为无驻留，1 为有驻留。

c. 候选节点执行准则：

有了初始优先级指标、对于 L0 级缓冲区依赖关系限制，本文确定 H 中节点的选取准则为：

在所有非 L0 级或 L0 级但对应类型的 Tag[0/1/2]=0 的节点中，选取优先级指标 $P(v_j)$

最小的节点，优先执行，若执行 L0 级节点 v_k ，则对控制 v_k 的其他 L0 级 ALLOC 节点 v_l (通过上述 BFS 得到)，令 $P(v_l) = P(v_l) - C$ ， C 为合适的正整数，即为 v_l 空间的申请节点提升优先级，以保证被控制的 L0 级节点 v_k 的相应驻留 L0 空间能够尽快释放，并同时维护 Tag。

在节点执行过程中实时维护 V_{stay} 、 $H = \{v_i, D_i = 0\}$ 等。

6.2 节点调度算法构建

基于上述贪心准则的构建，本文构建如下算法：

算法 1 节点调度算法

输入：有向图 $G = (V, E)$ 、常数 C

输出：调度序列 S 、最大缓存容量 V_{stay}

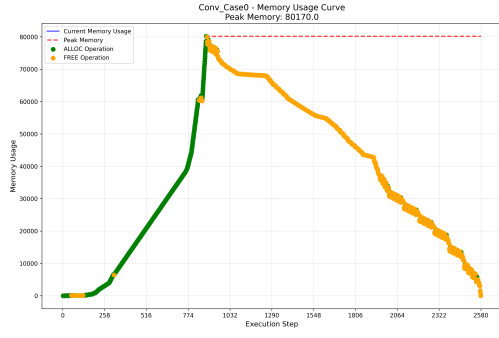
- 1: 设定初始优先级指标 $P(v_j) = \mathcal{M}(v_j)$ 、初始调度序列 S 、初始最大缓存容量 $V_{stay} = 0$ 、 $Tag[0, 1, 2] = [0, 0, 0]$
 - 2: **初始化操作**
 - a. 读入 $G = (V, E)$
 - b. 记录计算所有节点的初始入度 D_i ，生成初始候选节点集 $H = \{v_i, D_i = 0\}$
 - c. 读入计算图的边时，保存 ALLOC 和 FREE 节点的一一对应关系。
 - d. 通过 BFS 得到 L0 级缓冲区依赖关系，并保存。
 - 3: **根据 L0 级缓冲区依赖关系，更新图。**
 - 4: **while H 为非空集**
 - a. 在 H 中，在 v_m 为非 L0 或 v_m 为 L0 但对应类型的 $Tag==0$ 的节点中，找出中 P 最小的节点 v_m
 - i. 若 v_m 为非 L0 级节点，执行 v_m ，将 v_m 放入 S ，移出 H ，更新 V_{stay} ，并对该节点的后继节点入度减一。
 - ii. 若 v_m 为 L0 级节点，对应类型的 $Tag==0$ ，执行 v_m ，将 v_m 放入 S ，移出 H ，更新 V_{stay} ，对应类型的 $Tag[0/1/2]=1$ ，该节点的后继节点入度减一，并对控制该节点的其他 L0 级节点， $P(v_j) = P(v_j) - C$ 。
 - b. 检查 v_m 的后继节点的入度，若入度为 0，放入 H 。
-

根据上述算法，本文得到六个示例计算图的结果如表 6.4：

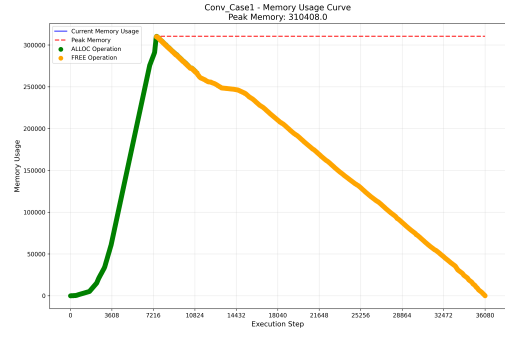
表 6.4 调度结果统计

示例图名称	节点数	$\max V_{stay}$ (字节)	调度顺序结果
FlashAttention_Case0	1716	26728.0	附件 1
FlashAttention_Case1	6952	9216.0	附件 2
Matmul_Case0	4160	14976.0	附件 3
Matmul_Case1	30976	34816.0	附件 4
Conv_Case0	2580	80170.0	附件 5
Conv_Case1	36086	310408.0	附件 6

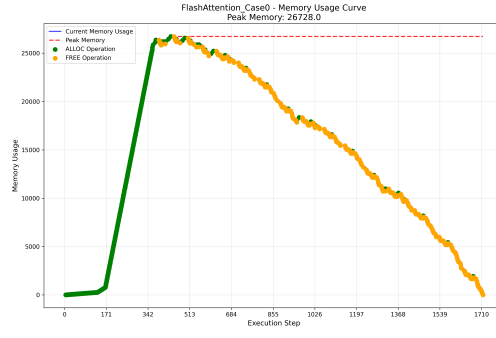
在计算过程中的 V_{stay} 变化如图 6.2



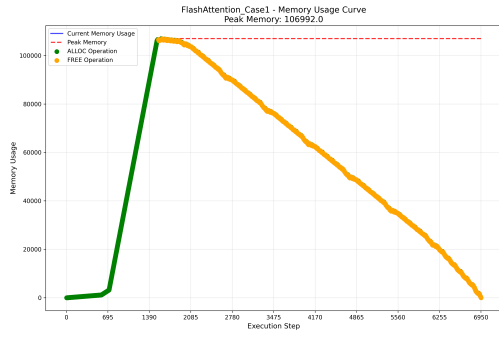
(a) FlashAttention_Case0



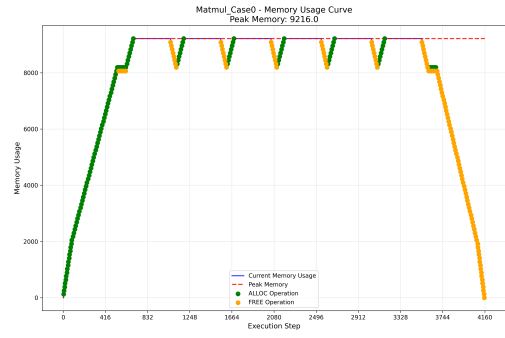
(b) FlashAttention_Case1



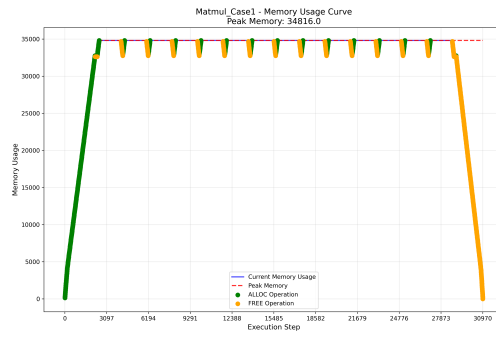
(c) Matmul_Case0



(d) Matmul_Case1



(e) Conv_Case0



(f) Conv_Case1

图 6.2 V_{stay} 随节点运行变化图

此外，本文算法的各案例性能指标如下图 6.3 所示，可见，本文算法耗时较短，且六个

案例全都满足对 L0 级缓冲区驻留的限制情况，得到的新 V_{stay} 较 V_{stay} 上界（即所有申请节点 Size 之和）提升均在 19% 以上，最高达到 47%，具有较为显著的提升。

案例名称	数据预处理 耗时 (s)	距离计 算耗时 (s)	ALLOC 节点数	FREE 节 点数	拓扑排 序耗时 (s)	L0A/B/ C 分配释 放匹配 情况	节点总 数	Vstay(UB +L1)	Vstay 上界	提升率
FlashAttention_Case0	0.14	0.23	176	176	0.12	匹配	1716	26728	35592	25%
FlashAttention_Case1	0.53	1.01	704	704	1.04	匹配	6952	106992	134288	20%
Matmul_Case0	0.32	0.64	1088	1088	0.75	匹配	4160	9216	16384	44%
Matmul_Case1	2.32	7.44	8448	8448	40.81	匹配	30976	34816	65536	47%
Conv_Case0	0.18	0.44	300	300	0.26	匹配	2580	80170	133031	40%
Conv_Case1	4.31	15.26	5330	5330	37.01	匹配	36086	310408	383112	19%
总耗时	159.86s									

图 6.3 各案例性能指标

6.3 算法时间复杂度分析

假设对 L0 级缓冲区的 ALLOC 操作共有 l 次 ($l \leq N$)，则数据读入部分、计算 D_i 、 H 、保存 ALLOC 和 FREE 一一对应的的时间复杂度共为 $O(N + M)$ ，多源 BFS 对每个源进行一次有向无环图的拓扑序 BFS，因此时间复杂度为 $O(l * M)$ ，在最坏情况下为 $O(N * M)$ 。对于主循环，外层循环每次会选取 V 中的一个顶点，因此外层循环次数为 N ，流程 (4a)、在 H 中移除节点、更新优先级的时间复杂度都为 $O(N)$ ，更新后继节点入度并在入度为 0 时放入 H 在两层循环中的总时间复杂度为 $O(\sum_{v \in V} N^+(v)) = O(M)$ 。因此，算法的时间复杂度为 $O(N^2 + NM)$ 。

7 问题二的建立与求解

7.1 缓存初始分配

本文首先使用 **Best-fit 算法** 建立初始缓存分配规则：

基于问题一给出的拓扑序 $\mathcal{S}$ ，依次执行当前计算图 DAG 中的各个节点。当执行到缓存管理节点 ALLOC 时，需要在对应类型缓存中申请相应大小地址；当执行到缓存管理节点 FREE 时，需要在在对应类型缓存中释放相应大小地址。每一块地址都有特定的物理地址偏移和长度作为唯一标识，即：(offset, size)。

本文为每个类型的缓存定义一个空闲列表 $\mathcal{L}_i$ 用来记录每次执行缓存管理节点后，当前空闲的缓存地址，是一个二元组的集合，即：

$$\mathcal{L}_{UB} = \{(\text{offset } 0, \text{size } 0), \dots, (\text{offset } k, \text{size } k)\} \quad (7.1)$$

且满足：

$$\sum_{i=1}^k \text{size } i \leq K \quad (7.2)$$

其中， K 为对应缓存类型的最大容量。每一个二元组表示空闲块的地址，这种表示方式是一一对应的。空闲列表随缓存上执行缓存管理节点和运行 **SPILL** 机制动态更新。

具体地，本文按照调度序列遍历**所有节点**，对于 **ALLOC** 节点，在相应类型空闲列表中找出 **Size** 大于等于所申请缓冲区 **Size** 的二元组，选取 **Size** 最小的二元组，将该二元组的 **offset** 作为该 **ALLOC** 节点的 **offset**，并更新空闲列表，将已分配的地址从空闲列表中删掉。对于 **FREE** 节点，可在内存中找到其对应的 **offset k**，于是将对应二元组 (**offset k**, **size k**) 加入空闲列表，并与其前后空闲二元组进行合并。

例如，对于 **UB** 类型的缓存，本文初始化 **UB** 的空闲列表为

$$\mathcal{L}_{UB} = \{(0, 1024)\} \quad (7.3)$$

当执行到某个 **ALLOC** 节点 v_k 时，假设其 **Size**=5，**Type**=**UB**，则此时，根据上述地址分配规则，为 v_k 申请 **UB** 缓存中 **offset** 为 0，**Size** 大小为 5 的空间，即 [0,4]，并更新空闲列表为：

$$\mathcal{L}_{UB} = \{(5, 1019)\} \quad (7.4)$$

按照调度顺序遍历所有节点，假设在某个 **FREE** 节点 v_f 之前的空闲列表为：

$$\mathcal{L}_{UB} = \{(5, 128) (200, 128) \dots\} \quad (7.5)$$

该 **FREE** 节点的对应 **offset** 为 0，**Size** 为 5，则将 (0, 5) 加入空闲列表，并与其前后空闲二元组进行合并，得到的新空闲列表为：

$$\mathcal{L}_{UB} = \{(0, 133) (200, 128) \dots\} \quad (7.6)$$

由此，算法完成了初始的缓存分配，在分配过程中，若遇到空闲列表无法满足所需地址大小的情况，则要进行缓存换入换出（即 **SPILL** 操作），以临时释放空间。

7.2 SPILL 操作

SPILL 操作是通过临时 **SPILL_OUT** 操作将部分缓冲区数据移至核外空间 **DDR** 上，释放出这部分缓存空间，并在后续计算需要时再通过 **SPILL_IN** 操作搬移回来的过程，此过程涉及到成对的新增操作节点 **SPILL_OUT** 和 **SPILL_IN**。

7.2.1 SPILL_OUT 数据的搬出

在进行 **SPILL_OUT** 操作时，将哪部分数据临时移动影响着释放的空间大小以及后续操作，因此，**SPILL_OUT** 操作对象的选取显得尤为重要。本文在 7.3 节中引入 **WCB 原则** 和 **MATCH 算法** 对此问题进行详细讨论。

7.2.2 SPILL_IN 数据的搬入

SPILL_IN 是指将核外数据重新加载至核内缓存地址区间,以供后续使用,由于 SPILL_IN 可将数据搬运至新的核内缓存空间,因此在结果上, SPILL_IN 操作等同于 ALLOC 操作,本文依旧按照 Best-fit 算法进行分配。

7.2.3 SPILL 操作节点的属性更新

在 SPILL 操作中,新增的两个操作节点的属性定义如表 7.5 所示。

表 7.5 新增操作节点属性设置规则

属性	设置规则
Id	从 N 开始自动递增。
Op	SPILL_OUT 或 SPILL_IN。
Pipe	SPILL_OUT 在 MTE3 单元上, SPILL_IN 在 MTE2 单元上。
Cycles	执行周期与当前目标缓冲区的大小有关,计算原则为: 若未被 COPY_IN 节点使用, $Cycles = 2 \times (Size \times 2 + 150)$ 若被 COPY_IN 节点使用, $Cycles = (Size \times 2 + 150)$ 。
Bufs	仅包含当前目标缓冲区 BufId 的列表。

对于 Cycles 的更新规则,若目标缓冲区在计算图中未被 COPY_IN 节点的使用,说明该缓冲区中的数据属于过程数据或结果数据,在 DDR 中未存放,因此 SPILL_IN 和 SPILL_OUT 分别执行数据搬出和搬入, $Cycles=2*(Size*2+150)$,若目标缓冲区在计算图中被 COPY_IN 节点的使用,说明该目标缓冲区中的数据为原始输入数据,在 DDR 中已经存放,仅需执行一次 SPILL_IN 数据搬入,SPILL_OUT 节点不产生核外存取, $Cycles=(Size*2+150)$ 。

7.2.4 SPILL 操作节点的计算图更新

在计算图中引入 SPILL 操作时,为保障数据搬移与原有计算流程的正确依赖关系,计算图上都会新增属性为 SPILL_OUT 和 SPILL_IN 这两个成对操作节点,且需按以下规则生成依赖边:

1. 生成三条基本依赖边:

首先建立 $ALLOC \rightarrow SPILL_OUT$ 、 $SPILL_OUT \rightarrow SPILL_IN$ 、 $SPILL_IN \rightarrow FREE$ 这三条基

基础依赖边。

2. 划分使用目标缓冲区的操作节点：

以 SPILL_OUT 节点在调度序列中的位置为界限，把当前计算图里所有使用到目标缓冲区的操作节点划分为“已执行”和“未执行”两部分。

对于“已执行”的操作节点 v_i ，生成 $v_i \rightarrow \text{SPILL_OUT}$ 的依赖边，确保已使用目标缓冲区的节点执行完毕后，才进行数据搬出操作。

对于“未执行”的操作节点 v_j ，生成 $\text{SPILL_IN} \rightarrow v_j$ 的依赖边，保证数据先搬入目标缓冲区后，未执行的节点才会使用该缓冲区数据。

假设在某个节点 v_j 执行时，因缓存不够需执行 SPILL 操作，加入 SPILL_OUT 和 SPILL_IN 这两个操作节点，从而导致 v_j 之后的的拓扑序列产生变化，此时仍然按照问题一中的节点调度更新后续拓扑序列，此部分将在 7.3 进行详细说明。

7.2.5 总额外数据搬运量

在 SPILL 操作中，会付出额外 DDR 访问数据量的增加，从而伴随高能耗。因此，在算法的评估指标中，需要考虑 SPILL 操作的代价，本题定义为总额外数据搬运量。

首先给出额外数据搬运量的概念：

额外数据搬运量：在一次 SPILL 操作中，引起的核外 DDR 访问数据量。其计算过程可根据目标缓冲区在计算图中是否被 COPY_IN 节点使用分成以下两种情况：

- Case1：一倍搬运量：

若目标缓冲区在计算图中被某个 COPY_IN 的操作节点使用，表示目标缓冲区数据来自 DDR，此时 SPILL_OUT 无需实际将该地址的数据搬回 DDR，仅删除或替换，因此 SPILL_OUT 不产生实际核外存取，仅 SPILL_IN 操作执行数据搬入操作。

- Case2：两倍搬运量：

若目标缓冲区在计算图中未被某个 COPY_IN 的操作节点使用，那么此时目标缓冲区存储的数据为结果数据或过程数据，且在未来某个时刻将被调用。若此时将其 SPILL_OUT 至 DDR 中，因 DDR 内尚不存在该数据，产生数据的核外存取。SPILL_IN 执行数据搬入操作，共计两倍数据搬运量。

此处，目标缓冲区在计算图中被某个 COPY_IN 的操作节点使用情况可根据缓冲区的 BufId 对应获取。

总额外数据搬运量为所有 SPILL 操作所带来的额外数据搬运量之和。

总额外数据搬运量定义了 SPILL 操作的代价，使其不能无限地执行。

7.3 缓存冲突管理方案

在某一缓存上，当前拓扑序 S 中执行到某个 ALLOC 节点时，需申请缓冲区，但当前已无足够缓存供其申请，则需要进行 SPILL 操作，即需要借助对某驻留的目标缓冲区实施

SPILL_OUT 操作，以制造足够使用的缓存空间，由于各个驻留的目标缓冲区的物理地址不同，长度不一，数据的使用时间不同，因此，本节重点讨论如何选取 SPILL_OUT 的操作对象。

为尽可能减小数据总额外搬运量，提高 SPILL 操作的效率，我们在选择时有以下原则：

- 在计算图中被某个 COPY_OUT 节点使用过的目标缓冲区优先选择。
- Size 大的目标缓冲区优先选择。
- 目标缓冲区的下一个操作节点在拓扑序中位置靠后的优先选择。（此处的拓扑序列为问题一不考虑所有 SPILL_IN 和 SPILL_OUT 节点的原始拓扑序，本文假设问题一生成的原始拓扑序列和加入 SPILL_IN 和 SPILL_OUT 节点的新拓扑序列差异在可接受的范围内）

7.3.1 WCB 评分模型

综合以上因素，我们引入 **WCB 评分模型 (Weighted Cost Benefit)**，对当前所有被占用缓冲区 b_j 进行打分。

在某个节点 v_n 执行时，需要引入 SPILL 操作，定义当前被占用所有缓冲区集合 $B = (b_1, b_2, \dots, b_k)$ ，各个被占用缓冲区 b_j 的 WCB 分数 $F(b_j)$ 为：

$$F(b_j) = w_1 \frac{c(b_j)}{\text{size}(b_j)} + w_2 \frac{1}{\text{next_use}_{\mathcal{S}}(b_j)} \quad (7.7)$$

其中， $\text{next_use}_{\mathcal{S}}(b_j)$ 表示在缓冲区 b_j 上执行的下一个节点与当前执行节点 v_n 在问题一拓扑序 $\mathcal{S}$ 中的距离，此距离通过拓扑序的编号计算。 w_1, w_2 为参数，默认 $(w_1, w_2) = (1, 1)$ 。 $c(b_j)$ 定义为：

$$c(b_j) = \begin{cases} 1, & \text{若缓冲区 } b_j \text{ 在计算图中被某个 COPY_OUT 节点使用,} \\ 2, & \text{若缓冲区 } b_j \text{ 在计算图中未被任何 COPY_OUT 节点使用.} \end{cases} \quad (7.8)$$

在公式 (7.7) 中，WCB 得分的计算公式给出，若某缓冲区的 size 较大、搬运量较小，表示未来使用位置与目前节点距离较远，则我们较倾向于选取该缓冲区，该缓冲区得分会较低。因此，WCB 得分实际上是对于缓冲区选取倾向程度的一种量化，并与缓冲区选取倾向程度呈负相关。

经计算 WCB 得分，可获得按 $F(b_j)$ 从小到大的缓冲区排序集 $B' = (b'_1, b'_2, \dots, b'_k)$ 。

7.3.2 SPILL_OUT 操作的目标缓冲区选取——MATCH 算法

在 WCB 得分的基础上，很自然的想到被 SPILL_OUT 执行的缓冲区应该为 WCB 得分最小的缓冲区，若该缓冲区中数据被搬出后仍无法满足需求，则对 WCB 得分次小的缓冲区进行第二次 SPILL 操作，依次类推。

但该方法无法保证先后两次选取的缓冲区能够构成连续的缓存地址空间，若不连续，则需要一直增加 **SPILL** 操作次数，增加数据搬运量。因此，本文提出 **MATCH 算法** (Minimum Adaptive Traverse Cache Hit, 最小自适应滑动缓存命中算法)。

如图 7.4，对处在某个 **ALLOC** 节点 v_e 的某类型缓存区，我们容易得到当前所有空闲缓冲区和所有占用缓冲区的起始地址偏移集合 $O = \{offset_i\}$ ，如图中箭头 a-i 所指位置。

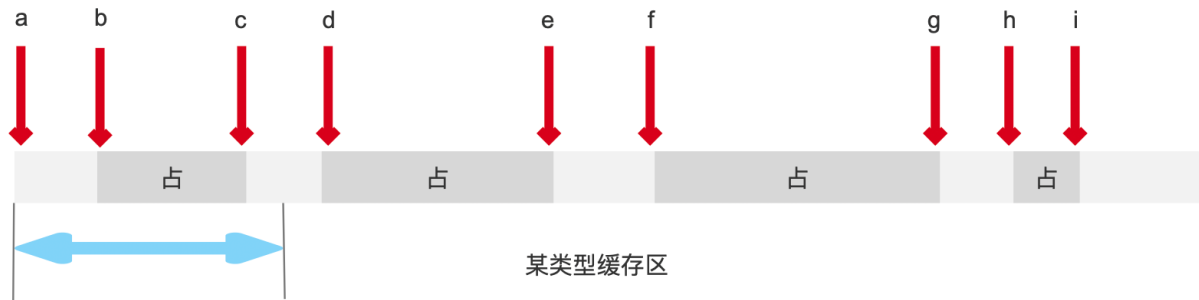


图 7.4 缓存占用示意图

在缓存死锁的情况下，对于 **ALLOC** 节点 v_e 的大小为 $size_e$ 的缓冲区申请需求，现有缓存区无法满足。

本文使用蓝色箭头区域表示 **ALLOC** 节点 v_e 可能申请的**虚拟缓冲区**，其中地址起点为某个已分配或未分配的缓冲区的地址偏移 (即 O 中元素)，大小为 $size_e$ 。对起点为 $Offset_i$ 的可能的虚拟缓冲区，本文将虚拟缓冲区地址记为 $A_i = [offset_i, offset_i + size_e]$ 。

本文使 $offset_i$ 遍历 O 中的每个地址偏移，并记录对应的虚拟缓冲区地址 A_i ，表示虚拟缓冲区可能的地址。同时，在此过程中记录与每个 A_i 相交的且被占用的缓冲区地址集合 B_i 。如图中所示，此时 A_i 遍历到第一个起始地址时，与其相交的被占用缓冲区为箭头 bc 之间的区域。

对于 B_i ，计算 B_i 中所有被占用缓冲区的 **WCB** 分数之和，记为 $\mathcal{F}_i$ 。对所有 A_i ，产生的所有 B_i ，找到最小的 $\mathcal{F}_i$ ，其对应的 B_i 即为所有要执行 **SPILL_OUT** 操作的缓冲区集合，使用 **SPILL_OUT** 操作将集合中对应的缓冲区内数据依次搬入 **DDR**。

由此，我们找到了在缓存冲突时，**SPILL_OUT** 的目标缓冲区。该方法在遵循 **WCB** 分数小优先选取的原则上，保证了每次腾出的缓冲区是连续的地址空间。

算法 2 MATCH 算法

输入: 计算图 $\mathcal{G} = (V, E)$ 、拓扑序 $\mathcal{S}$ 、当前 ALLOC 节点 v_e 、其缓冲区申请大小 $size_e$ 、已分配或未分配缓冲区的地址偏移集合 O

输出: SPILL_OUT 操作的目标缓冲区集合 $\mathcal{B}_i$

- 1: 遍历 O 中的每个地址偏移 $offset_i$ ，计算虚拟缓冲区地址 $A_i = [offset_i, offset_i + size_e]$ 。
 - 2: 记录与 A_i 相交的被占用缓冲区地址集合 $\mathcal{B}_i$ ，并计算 $\mathcal{B}_i$ 中所有缓冲区 WCB 分数之和 $\mathcal{F}_i$
 - 3: 找出最小 $\mathcal{F}_i$ 及其对应的 $\mathcal{B}_i$ ，即为目标缓冲区集合 $\mathcal{B}_i$
-

7.4 调度算法综述

综上所述，本文将所构建的调度算法如图 7.5 所示：

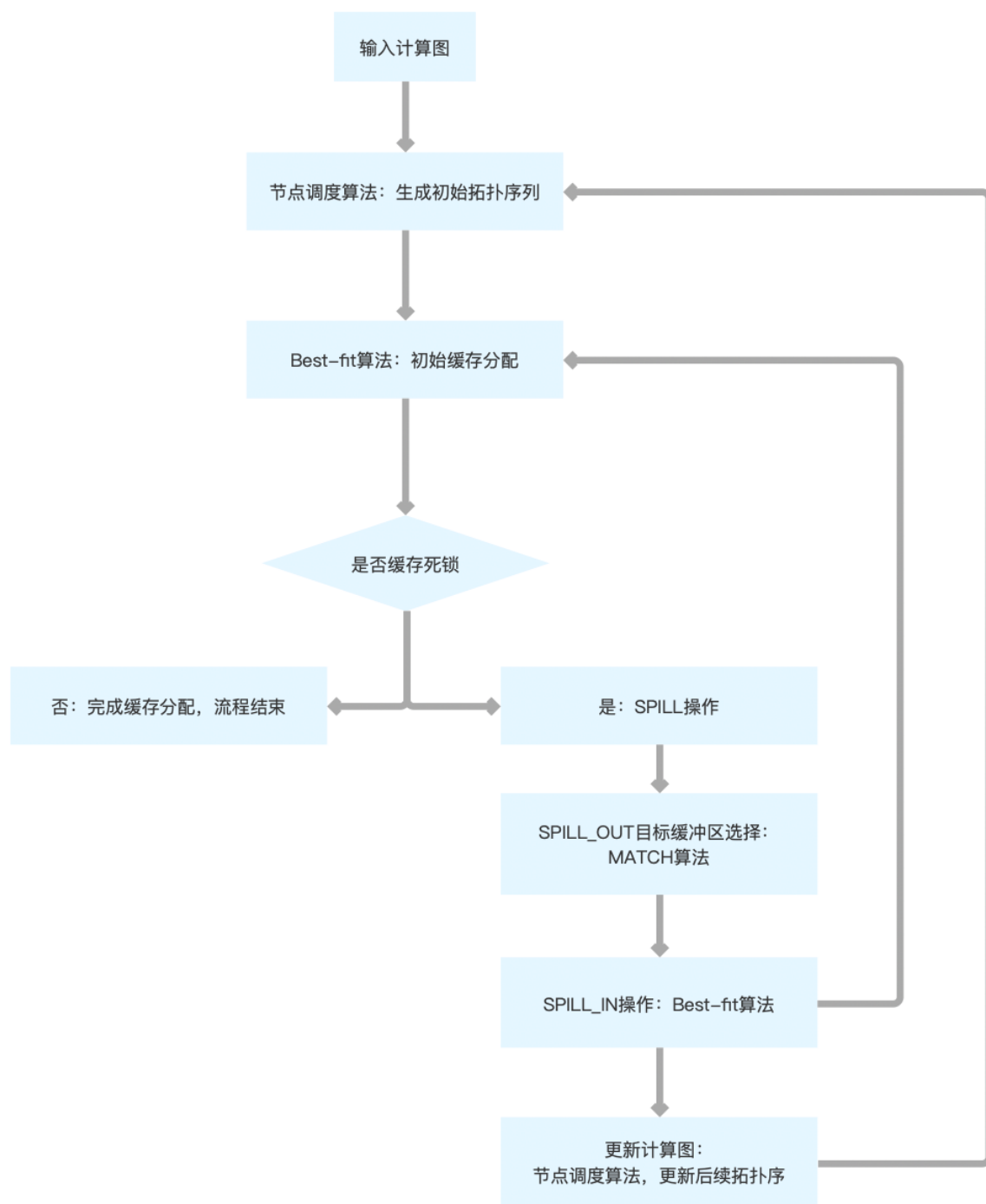


图 7.5 调度算法

给定一个计算图，本文通过节点调度算法得到初始节点的拓扑序列，随后根据 Best-fit 算法对进行初始缓存分配，若遇到缓存死锁，则进行 SPILL 操作。SPILL 操作时，SPILL_OUT 节点的选择使用 MATCH 算法，SPILL_IN 仍采用 Best-fit 算法，在 SPILL 的计算图更新过程中使用节点调度算法对 SPILL 操作后续节点拓扑序进行动态更新。

7.5 算法应用结果

依据上述调度算法，本文对六个示例图进行调度方案的分配，得到的结果如表 7.6：

表 7.6 调度算法结果

示例图名称	节点数	总额外数据搬运量	SPILL 次数
FlashAttention_Case0	1716	48896.0	191
FlashAttention_Case1	6952	216864.0	1078
Matmul_Case0	4160	24576.0	96
Matmul_Case1	30976	122880.0	480
Conv_Case0	2580	256576.0	393
Conv_Case1	36086	757968.0	5805

由图可见，因额外数据搬运量与节点数有一定的相关关系，观察发现，在节点数不大的情况下，额外数据搬运量在 Conv 类型的两个示例图上显著较大，通过观察示例计算图特点（详见 10.2 节），发现 Conv 类型的两个计算图的缓存申请 Size 具有较大的差异，从而使得缓存会产生更大的碎片化，导致需要进行较多次的 SPILL 操作，针对这种问题，本文提出了通过分层管理的优化方案，详见 10.2 节。

8 问题三的建立与求解

8.1 流水排布

由于前两问关于各类操作节点的执行过程中，由于是按照调度顺序 S 线性执行的，因此只要拓扑序上某一节点尚未执行完成，位于其位置之后的节点都无法执行，如果使用流水图表示，即：所有执行单元上的不同操作节点，投影到时间轴上是互不交的，如图 8.6。

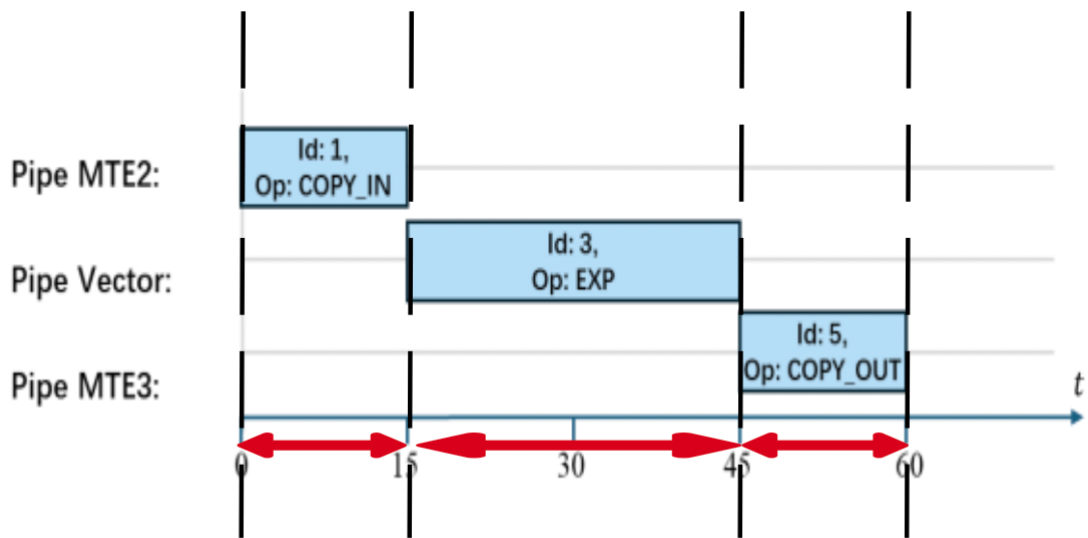


图 8.6 调度顺序时序流水图

但是若考虑不同硬件单元实则可以并行执行，若满足其执行的前置条件。位于不同执行单元上的各个节点那么可以减少总执行时间。

8.2 计算图相关时钟设定

本问题需要将时钟的基本设定引入到计算图和拓扑序中。

由题目信息可知，对于计算图上的节点 v_i ，起始时间记为 $S(v_i)$ ，结束时间记为 $E(v_i)$ （单位为时钟周期），存在以下执行耗时公式：

$$E(v_i) = S(v_i) + \text{cycles}(v_i) \quad (8.9)$$

其中 cycles 表示执行该节点所需要的时钟数。对于缓存管理节点， $\text{cycles} = 0$ ，因此可以认为在调度序列读取到缓存管理节点时，可以瞬间执行完成。而对于操作节点，则在执行时都有一定的时钟数消耗，其中最为特殊的是 **SPILL** 节点，它们在执行时消耗的时钟数与本身占用的物理地址大小有关，即：

$$\begin{aligned} \text{Cycles of SPILL_OUT op} &= \text{Size} * 2 + 150 \\ \text{Cycles of SPILL_IN op} &= \text{Size} * 2 + 150 \end{aligned} \quad (8.10)$$

其他节点也有各自不同的时钟消耗，我们将这些操作节点的相关数据总结在如下表格中。从表8.7中可以得到如下结论：

1. 仅仅是 **SPILL** 节点的时钟数与物理地址大小相关，其他操作节点均为常数；

表 8.7 各类操作节点时钟数消耗情况

节点名称	执行单元	消耗时钟数
SPILL_OUT	MTE3	$size*2+150$
SPILL_IN	MTE2	$size*2+150$
COPY_IN	MTE2	c_1
COPY_OUT	MTE3	c_2
FIXP	MTE3	c_2
MOVE	MTE1	c_4

2. 当优化任务总执行时间时，SPILL 节点的物理地址长度是重要考量因素，因为地址长度更长的缓存块在执行 SPILL 操作时花费的时钟数越多，这会导致 MTE2/3 执行单元长期占用。

除了计算图的节点需要更新，还需要向图中添加新的有向边。要遵循问题二中的原则之外，还要增加计算缓存服用依赖，即如果缓冲区 b 复用了缓冲区 a 的地址空间，那么要添加一条从 a 的 FREE 节点指向 b 的 ALLOC 节点的依赖边。这里复用指的是地址空间的交集不为空，但并非同一时刻存在，因此可能存在一个 FREE 节点指向多条 ALLOC 节点的情况，如图 8.7。

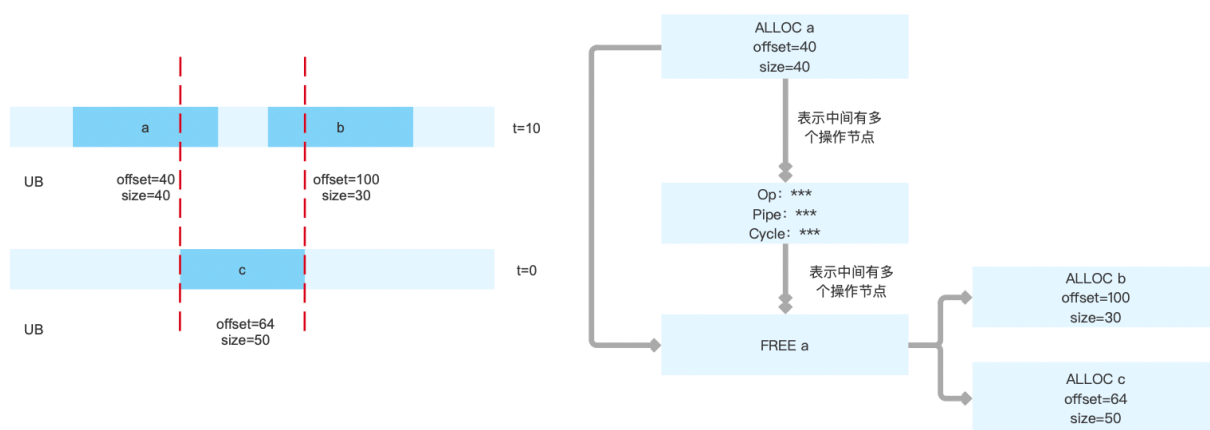


图 8.7 同一缓存区先后交集非空时依赖示意图

由图可知，若存在地址复用，无论复用时有多少个新地址空间在地址上和旧地址有交集，都需要存在到新空间的指向边。

8.3 基于空闲压缩的 ASAP 方法

8.3.1 ASAP 方法的有效性

在流水排布的模式里，只要存在某个操作节点，其满足以下三个条件：

- 已无执行依赖：即在计算图 DAG 中的所有前向依赖节点都被执行完成；
- 执行单元可用：该操作节点所需的执行单元（如 MTE3、MTE2 等）不被其他操作节点占用；
- 物理地址已分配：即操作节点执行时所依赖的地址已分配。

那么该操作节点是可以开始执行的，完全不同与先前单线执行的模式，称上述条件为前置条件。

ASAP 的思想是：单线执行的模式往往忽略了具体各个执行单元的工作情况，某些操作单元在某些时钟数区间上可能存在闲置，因为按照拓扑序线性执行的思想，在某一时钟最多有一个操作节点占用，因此其他执行单元都是无任务的，但很可能有操作节点同时满足上述三个前置条件，那么此时可以立即开始执行，从时钟数上是提前的，从流水图的时间轴上看就像是向左进行滑动一样。下面将简单论述这种思想可行的原因。

1. 执行依赖：按照计算图上的制约，前依赖节点全部执行完才可以执行当前操作节点，这是并行调度也必须遵守的规则。而第二问中的计算图结构保证其成立，且第三问的计算图添加了更多的边，意味着执行依赖关系更严格，这种执行依赖是不会被打破的。
2. 物理地址分配：拓扑序总是要求先 ALLOC 分配地址再执行操作节点，而地址是复用的，在计算图中表现为有 FREE 指向 ALLOC 的依赖边。且缓存分配方案天然禁止了同一生命周期上出现物理地址的重叠。

综上，ASAP 方法是具有可行性的。

8.3.2 实现原理

ASAP 方法目的是将当前操作节点的时钟滑动至前置条件均满足的最早时钟。这一时钟受制于前驱完成时间、执行单元可用、地址可用，因此计算节点的最早启动时间要取这三者的最大时钟值，即：

$$\text{early}(v) = \max \left(\max_{u \in \text{pred}(v)} E(u), \text{pipe_last}[\text{pipe}(v)], \text{addr_free}(\text{buf}(v)) \right)$$

其中 $\text{early}(v)$ 是节点的最早启动时间。 $\text{addr_free}(\text{buf}(v))$ 是该缓冲区被 FREE 的时钟。依据公式 (8.3.2) 可以计算得到该节点的新启动时间 $S'(v_i)$ 和结束时间 $E'(v_i)$

$$S_{\text{new}}(v) = \text{early}(v) \quad (8.11)$$

$$E_{\text{new}}(v) = S_{\text{new}}(v) + \text{Cycles}(v) \quad (8.12)$$

这种方法并不需要改变原有调度序列，只是在保证不违背原则的情况下，让满足前置条件的节点不必单线等待拓扑序的调用，直接开始读取。尽可能压缩了各执行单元的空闲时间。

8.4 求解结果

按照 ASAP 方法，重新计算总任务时长，并将得到的优化后的结果同未优化相比较，呈现在??中：

表 8.8 优化算法结果

示例图名称	节点数	流水总时钟	流水总时钟（左滑后）	时间下降
FlashAttention_Case0	1716	43620	25600	41.31%
FlashAttention_Case1	6952	171448	104960	38.78%
Matmul_Case0	4160	94025	65536	30.30%
Matmul_Case1	30976	43620	25600	31.36
Conv_Case0	2580	371775	348677	6.21%
Conv_Case1	36086	644592	409566	36.46%

从以上结果中，可以推出以下结论：

1. 应用 ASAP 并行执行方法后，各个数据集都出现了一定的执行时钟数下降，但是效果好坏不尽相同。如对于 FlashAttention_Case0 数据集，则效果表现优异，可以下降约 30% 到 40%，说明原先数据在执行时因调度顺序和单线执行造成的阻塞较为严重。而如 Conv_Case0 数据集则提升效果不尽明显。
2. ASAP 方法对于压缩不同执行单元的空闲时钟，降低任务总执行时间有积极意义。

9 灵敏度分析

在问题二的 WCB 打分模型中，涉及到了参数 w_1 、 w_2 ，为验证模型的灵敏度，本节进行参数扫描，参数选取为 $[0.5, 1, 2] \times [0.5, 1, 2]$ ，结果如图 9.8、图 9.9、图 9.10所示：

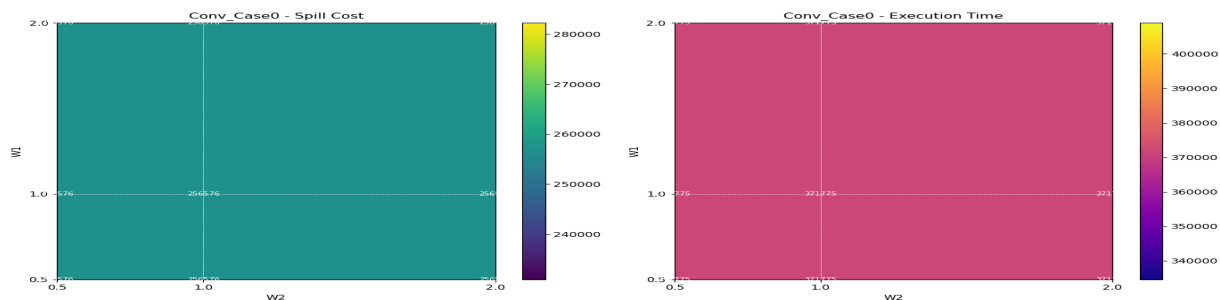


图 9.8 Conv_Case0_heatmap

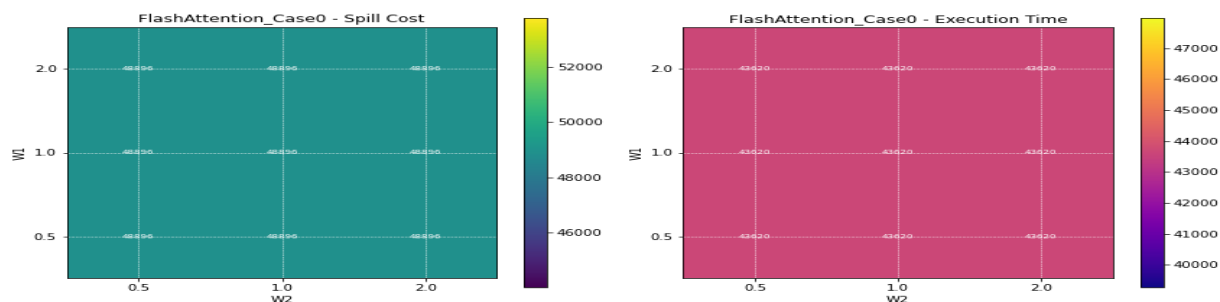


图 9.9 FlashAttention_Case0_heatmap

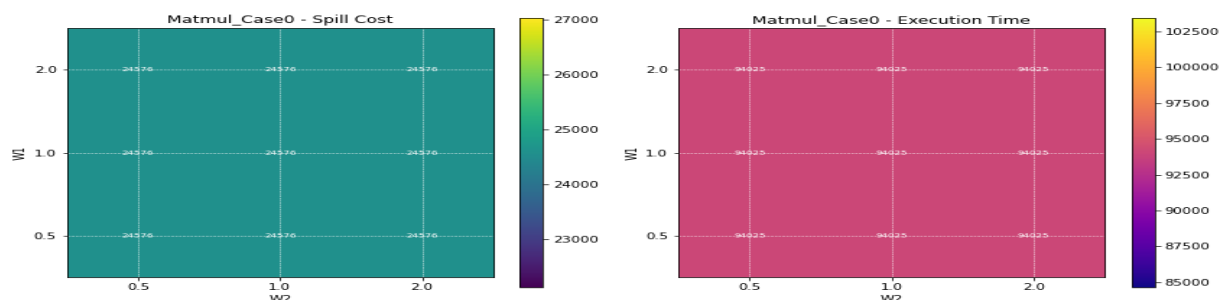


图 9.10 Matmul_Case0_heatmap

我们发现：

1. 在不同的参数组合下，得到数据搬运量和运行时钟数并无显著差异（或者说完全不变），这同时对六个数据集成立；
2. 我们的方法并没有针对某一特定算子的计算图结构进行优化，从效果上具有普适性。也说明模型是稳健的。

10 模型评价

10.1 模型优缺点

10.1.1 模型优点

1. 在问题一中，本文所构建的算法性能较好，且较缓存容量上界有显著的提升，算法充分考虑了 L0 级缓冲区的限制，保证了 L0 级缓冲区同类同时最多只有一个驻留的限制条件。
2. 在问题二中，本文首先通过 Best-fit 算法进行初始容量分配，同时使用 WCB 和 MATCH 算法为 SPILL 操作选择合适的缓冲区，保证了 SPILL 操作的效率，在尽量减少 SPILL 操作的次数的基础上最大化 SPILL 操作的效果。

10.1.2 模型不足

在问题二的初始缓存分配方案中，本文按照拓扑序列在整个缓存空间进行扫描，顺序排布，可能会导致缓存中出现较多的缝隙，即缓存的碎片化。

10.2 模型改进

在问题二中，由上文分析，可能会出现缓存碎片化的问题。这本质上是由于顺序扫描造成节点的 Size 变化较大，ALLOC 和 FREE 的并未按照节点 Size 大小进行分级管理。

因此，本文提出对缓存空间进行分级管理，划分公用和私用缓存空间，对于 Size 差异不大的 ALLOC 节点，统一申请私用空间的地址块，以实现私用空间的充分利用，对于 Size 差异较大的 ALLOC 节点，申请公用空间，先为 Size 较大的节点申请空间，随后为 Size 较小的节点申请空间，以填补公用空间的缝隙。这样根据节点 Size 进行分层管理的方法，是建立在计算图中 ALLOC 节点 Size 的分布较为集中的基础上的。

这是因为，在典型的 Matmul 计算图、FlashAttention 计算图、CONV 计算图中，都涉及到矩阵乘法，而矩阵乘法通常会涉及到矩阵分块操作，分块操作倾向于将矩阵分成大小相近的块，这就导致了 ALLOC 节点的 Size 大小分布较为集中，甚至出现相同的情况。本文将给出的六个示例图的 ALLOC 节点进行统计如图 10.11（仅统计 L1 类型），发现结论确实如上所述。

案例名称	ALLOC且L1节点总数	Size均值	Size最大	Size最小	0.75分位数	0.25分位数
FlashAttention_Case0	40	256	256	256	256	256
FlashAttention_Case1	112	256	256	256	256	256
Matmul_Case0	128	128	128	128	128	128
Matmul_Case1	512	128	128	128	128	128
Conv_Case0	531	250.53	1536	1	144	64
Conv_Case1	6683	57.33	256	1	64	32

图 10.11 示例图 ALLOC 节点 Size 统计

例如图 10.12，可以对 L1 级 ALLOC 节点按照 Size 大小进行统计，从而分区申请。

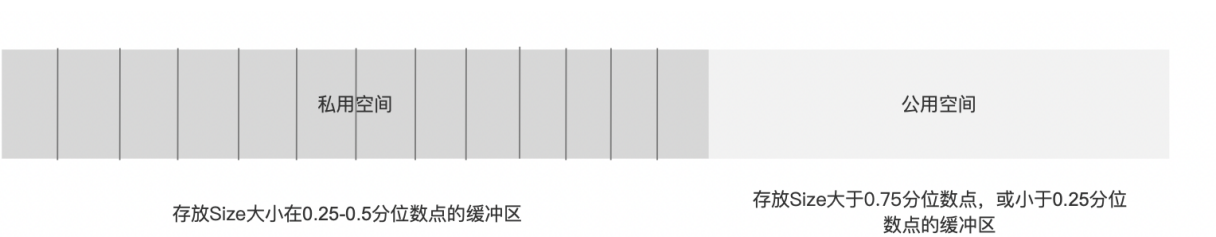


图 10.12 缓冲区分级管理示例

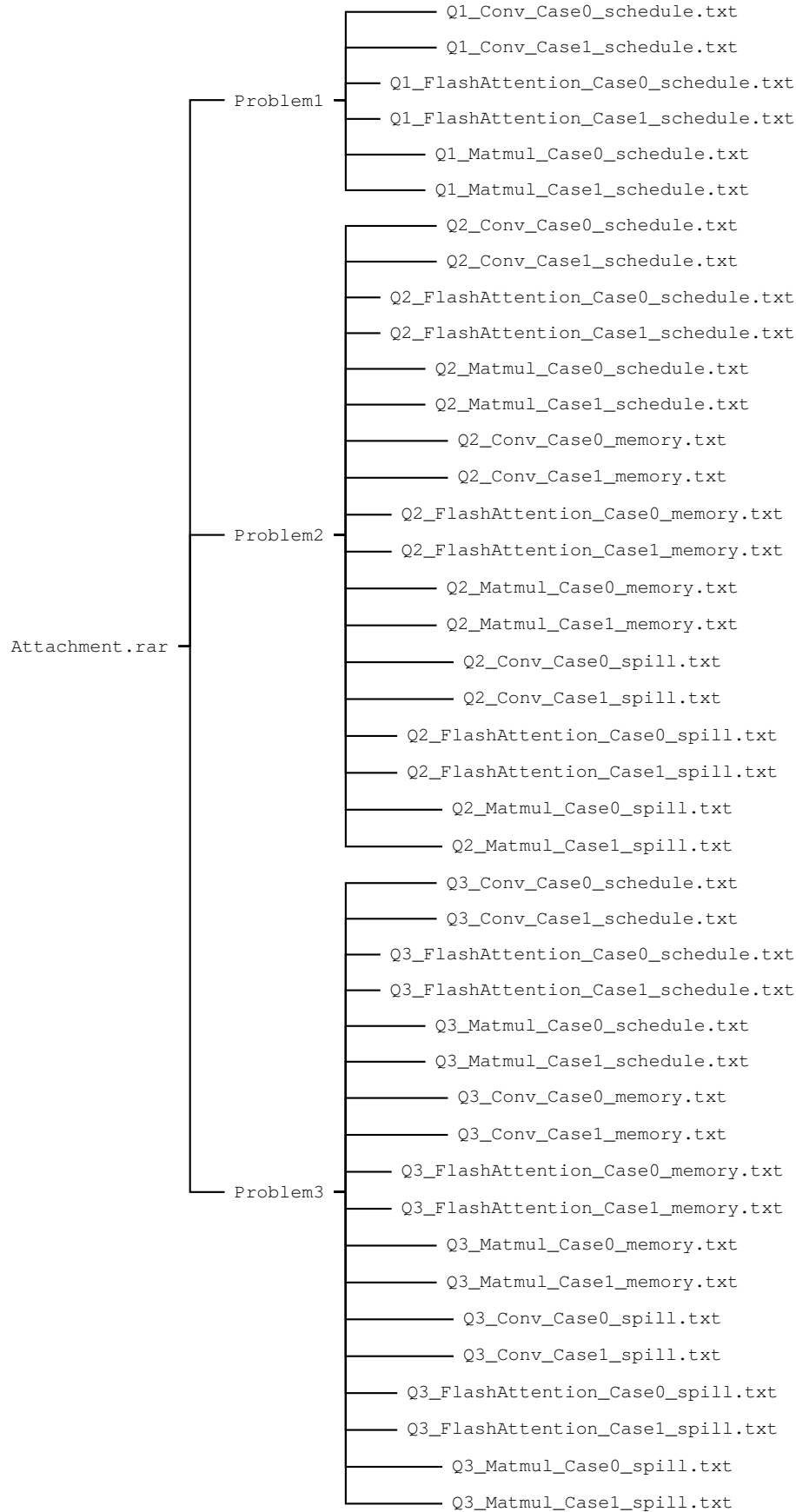
如图，根据数据特点，在私用空间分块存在大小在 0.25-0.5 分位数点之间缓冲区，在公用空间存在大小大于 0.75 或小于 0.25 分位数点的缓冲区，实现按照缓冲区大小进行分级管理。

参考文献

- [1] “华为杯”第二十二届中国研究生数学建模竞赛论文格式规范
- [2] D.Lea, “A Memory Allocator.” 2000. [Online]. <https://gee.cs.oswego.edu/dl/html/malloc.html>.
- [3] L. Marchal, H. Nagy, B. Simon and F. Vivien, ”Parallel Scheduling of DAGs under Memory Constraints,” 2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Vancouver, BC, Canada, 2018, pp. 204-213, doi: 10.1109/IPDPS.2018.00030.
- [4] 刘秋月. 基于 Alluxio 的跨域数据高效存取方法研究 [D]. 齐鲁工业大学,2025.DOI:10.27278/d.cnki.gsdqc.2025.000857.
- [5] 晋高成, 李丕丁. 异构多核调度算法研究综述 [J]. 软件导刊,2021,20(02):238-243.
- [6] 阳王东, 王昊天, 张宇峰, 等. 异构混合并行计算综述 [J]. 计算机科学,2020,47(08):5-16+3.

附录 A 支撑材料文件

1.1 Attachment.rar



附录 B Python 源程序

B.1 第1问程序

Q1.py

```
#本程序及代码是在人工智能工具辅助下完成的，使用了Deepseek模型
'''
模型版本：DeepSeek-V3.1-Terminus
开发机构：杭州深度求索人工智能基础技术研究有限公司
颁布日期：2025年9月22日
'''

import pandas as pd
import os
import numpy as np
import networkx as nx
import time
import matplotlib.pyplot as plt
from collections import deque

def read_csv_optimized(case):
    """读取节点和边数据，向量化计算Cost"""
    nodes = pd.read_csv(f"{case}_Nodes.csv")
    edges = pd.read_csv(f"{case}_Edges.csv")

    def calculate_cost(row):
        """计算节点Cost"""
        if row['Type'] in {'UB', 'L1'}:
            if row['Op'] == 'ALLOC':
                return row['Size']
            elif row['Op'] == 'FREE':
                return -row['Size']
        return 0

    nodes['Cost'] = nodes.apply(calculate_cost, axis=1).astype(
        int)

    if "NodeId" in nodes.columns:
        nodes.set_index("NodeId", inplace=True)
```

```

    return nodes, edges

def build_dag_fast(edges):
    """快速构建有向无环图"""
    edge_tuples = [(int(r.StartNodeId), int(r.EndNodeId)) for _,
                    r in edges.iterrows()]
    return nx.from_edgelist(edge_tuples, create_using=nx.DiGraph)

def precompute_dominating_effects(dominating_relation, row_index,
    rev):
    """
    预处理支配关系效果
    dominating_effects[node] = 被node支配的节点列表
    """
    dominating_effects = {}
    n = len(row_index)

    for i in range(n):
        dominator_node = row_index[i]
        dominating_effects[dominator_node] = []

        for j in range(n):
            if dominating_relation[i][j] == 1:
                dominated_node = row_index[j]
                dominating_effects[dominator_node].append(
                    dominated_node)

    return dominating_effects

def greedy_topo_optimized(nodes, graph, dominating_relation,
    row_index, rev, index):
    """优化版贪心拓扑排序"""
    # 预处理支配关系
    dominating_effects = precompute_dominating_effects(
        dominating_relation, row_index, rev)

```

```

# 预先提取属性到字典（避免频繁pandas操作）
node_attrs = {}
for nid, row in nodes.iterrows():
    node_attrs[nid] = {
        'Op': row['Op'],
        'Type': row['Type'],
        'Cost': row['Cost']
    }

# 初始化入度和零入度节点队列
in_degree = {n: graph.in_degree(n) for n in graph.nodes}
zero_degree_nodes = [n for n in graph.nodes if in_degree[n]
    == 0]

execution_order = []
l0a_count, l0b_count, l0c_count = 0, 0, 0

while zero_degree_nodes:
    # 选择Cost最小的节点
    min_cost = float('inf')
    selected_node = None

    for node in zero_degree_nodes:
        attr = node_attrs[node]
        if attr['Cost'] < min_cost:
            # 检查L0类型分配限制
            if attr['Op'] == "ALLOC":
                if (attr['Type'] == "L0A" and l0a_count >= 1)
                    or \
                    (attr['Type'] == "L0B" and l0b_count >= 1)
                    or \
                    (attr['Type'] == "L0C" and l0c_count >= 1)
                    :
                    continue
            min_cost = attr['Cost']
            selected_node = node

    current_node = selected_node

```

```

current_attr = node_attrs[current_node]

# 更新L0类型计数
if current_attr['Op'] == "ALLOC":
    if current_attr['Type'] == "L0A":
        l0a_count += 1
    elif current_attr['Type'] == "L0B":
        l0b_count += 1
    elif current_attr['Type'] == "L0C":
        l0c_count += 1
elif current_attr['Op'] == "FREE":
    if current_attr['Type'] == "L0A":
        l0a_count -= 1
    elif current_attr['Type'] == "L0B":
        l0b_count -= 1
    elif current_attr['Type'] == "L0C":
        l0c_count -= 1

# 处理支配关系效果
if (current_attr['Op'] == "ALLOC" and
    current_attr['Type'] in {"L0A", "L0B", "L0C"} and
    current_node in dominating_effects):

    for affected_node in dominating_effects[current_node]:
        node_attrs[affected_node]['Cost'] = -1

# 更新后继节点入度
for successor in graph.successors(current_node):
    in_degree[successor] -= 1
    if in_degree[successor] == 0:
        zero_degree_nodes.append(successor)

execution_order.append(current_node)
zero_degree_nodes.remove(current_node)

return execution_order

```

```

def calculate_memory_peak(nodes, execution_order):
    """计算内存使用峰值"""
    current_memory = peak_memory = 0
    l0a_count, l0b_count, l0c_count = 0, 0, 0
    allocation_match = True

    for node_id in execution_order:
        row = nodes.loc[node_id]

        # UB和L1类型的内存分配/释放
        if row.Op == "ALLOC" and row.Type in {"UB", "L1"}:
            current_memory += row.Size
            peak_memory = max(peak_memory, current_memory)
        elif row.Op == "FREE" and row.Type in {"UB", "L1"}:
            current_memory -= row.Size

        # L0类型的计数跟踪
        elif row.Op == "ALLOC":
            if row.Type == "L0A": l0a_count += 1
            elif row.Type == "L0B": l0b_count += 1
            elif row.Type == "L0C": l0c_count += 1
        elif row.Op == "FREE":
            if row.Type == "L0A": l0a_count -= 1
            elif row.Type == "L0B": l0b_count -= 1
            elif row.Type == "L0C": l0c_count -= 1

        # 检查L0分配是否匹配
        if l0a_count > 1 or l0b_count > 1 or l0c_count > 1:
            allocation_match = False

    allocation_status = "匹配" if allocation_match else "不匹配"
    print(f"L0A/B/C 分配与释放{allocation_status}")

    return peak_memory

def save_execution_schedule(case, execution_order, peak_memory):
    """保存执行调度结果"""
    print(f"{case:<20} 节点数: {len(execution_order):<6} 峰值(UB

```

```

+L1): {peak_memory:<6}")

with open(f"{case}_schedule.txt", "w") as f:
    for node_id in execution_order:
        f.write(f"{node_id}\n")

def calculate_alloc_free_distances_optimized(nodes, edges, graph,
case_name):
    """优化版计算ALLOC L0节点到FREE节点的距离"""
    # 快速构建节点信息
    node_info = {
        nid: {'Op': row['Op'], 'Type': row['Type']}
        for nid, row in nodes.iterrows()
    }

    # 找出集合A (ALLOC L0节点) 和集合B (相连的FREE节点)
    alloc_nodes = [
        nid for nid, row in nodes.iterrows()
        if row['Op'] == 'ALLOC' and row['Type'] in {'L0A', 'L0B',
            'L0C'}
    ]

    free_nodes = set()
    for _, edge in edges.iterrows():
        start_node = int(edge.StartNodeId)
        end_node = int(edge.EndNodeId)

        if (start_node in node_info and end_node in node_info and
            start_node in alloc_nodes and node_info[end_node]['Op'
                '] == 'FREE'):
            free_nodes.add(end_node)

    sorted_alloc_nodes = sorted(alloc_nodes)
    sorted_free_nodes = sorted(free_nodes)

    # 多源BFS计算距离
    distance_matrix = multi_source_bfs_distances(graph,
        sorted_alloc_nodes, sorted_free_nodes)

```

```

    return distance_matrix, sorted_alloc_nodes, sorted_free_nodes

def multi_source_bfs_distances(graph, sources, targets):
    target_to_index = {target: j for j, target in enumerate(
        targets)}
    source_to_index = {source: i for i, source in enumerate(
        sources)}

    distance_matrix = np.full((len(sources), len(targets)), -1,
        dtype=int)

    # 预先计算所有目标节点的可达性
    target_set = set(targets)

    for i, source in enumerate(sources):
        # 使用字典记录到各个目标节点的最短距离
        min_distances = {target: float('inf') for target in
            targets}

        visited = set()
        queue = deque([(source, 0)])
        visited.add(source)

        if source in target_set:
            min_distances[source] = 0

        while queue:
            current, dist = queue.popleft()

            # 如果已经找到所有目标节点，可以提前终止
            if all(d != float('inf') for d in min_distances.
                values()):
                break

            for neighbor in graph.successors(current):
                if neighbor not in visited:
                    visited.add(neighbor)

```

```

        new_dist = dist + 1

        if neighbor in target_set:
            if new_dist < min_distances[neighbor]:
                min_distances[neighbor] = new_dist

        queue.append((neighbor, new_dist))

    # 更新距离矩阵
    for target, min_dist in min_distances.items():
        if min_dist != float('inf'):
            j = target_to_index[target]
            distance_matrix[i, j] = min_dist

    return distance_matrix

def plot_alloc_size_histogram(nodes, case_name):
    """绘制ALLOC操作的Size分布直方图"""
    alloc_nodes = nodes[nodes['Op'] == 'ALLOC']

    if alloc_nodes.empty:
        print(f"{case_name}: 没有找到ALLOC操作")
        return

    sizes = alloc_nodes['Size'].values

    plt.figure(figsize=(10, 6))
    n, bins, patches = plt.hist(sizes, bins=30, alpha=0.7, color=
        'skyblue', edgecolor='black')

    plt.title(f'{case_name} - ALLOC Size Distribution', fontsize
        =14)
    plt.xlabel('Size', fontsize=12)
    plt.ylabel('Frequency', fontsize=12)
    plt.grid(axis='y', alpha=0.75)

    # 添加统计信息
    stats_text = f'Total ALLOC: {len(sizes)}\nMin Size: {np.min(

```

```

        sizes)}\nMax Size: {np.max(sizes)}\nMean Size: {np.mean(
        sizes):.2f}'
plt.text(0.02, 0.95, stats_text, transform=plt.gca().
        transAxes,
            fontsize=10, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha
                    =0.5))

plt.savefig(f'{case_name}_alloc_size_histogram.png', dpi=300,
        bbox_inches='tight')
plt.close()

def main():
    os.chdir(os.path.dirname(os.path.abspath(__file__)))
    """主函数"""
    cases = [
        "FlashAttention_Case0", "FlashAttention_Case1",
        "Matmul_Case0", "Matmul_Case1",
        "Conv_Case0", "Conv_Case1"
    ]

    print("==== 内存调度优化算法 ====")
    start_time = time.time()

    for case in cases:
        print(f"\n=== 处理 {case} ===")

        # 数据读取和预处理
        data_start = time.time()
        nodes, edges = read_csv_optimized(case)
        graph = build_dag_fast(edges)
        data_time = time.time() - data_start
        print(f"数据预处理耗时: {data_time:.2f}s")

        # 距离计算
        dist_start = time.time()
        distances, alloc_nodes, free_nodes =
            calculate_alloc_free_distances_optimized(

```

```

        nodes, edges, graph, case
    )
    print(distances)
    dist_time = time.time() - dist_start
    print(f"距离计算耗时: {dist_time:.2f}s")
    print(f"ALLOC节点数: {len(alloc_nodes)}, FREE节点数: {len(
        free_nodes)}")

    # 构建索引映射
    index_map = np.zeros(np.max(free_nodes) + 1, dtype=int)
    for i, node_id in enumerate(free_nodes):
        index_map[node_id] = alloc_nodes[i]

    # 处理距离关系并添加边
    for i, alloc_node in enumerate(alloc_nodes):
        for j, free_node in enumerate(free_nodes):
            if distances[i][j] > 1:
                distances[i][j] = 1
                alloc_type = nodes.loc[alloc_node]['Type']
                free_type = nodes.loc[free_node]['Type']

                if alloc_type == free_type:
                    graph.add_edge(alloc_node, index_map[
                        free_node])
            else:
                distances[i][j] = 0

    # 构建反向索引
    reverse_index = np.zeros(np.max(alloc_nodes) + 1, dtype=
        int)
    for i, node_id in enumerate(alloc_nodes):
        reverse_index[node_id] = i

    # 拓扑排序
    topo_start = time.time()
    execution_order = greedy_topo_optimized(
        nodes, graph, distances, alloc_nodes, reverse_index,
        index_map
    )

```

```

topo_time = time.time() - topo_start
print(f"拓扑排序耗时: {topo_time:.2f}s")

# 计算结果并保存
peak_memory = calculate_memory_peak(nodes,
    execution_order)
save_execution_schedule(case, execution_order,
    peak_memory)

# 可选: 绘制直方图
# plot_alloc_size_histogram(nodes, case)

total_time = time.time() - start_time
print(f"\n总耗时: {total_time:.2f}s")

if __name__ == "__main__":
    main()

```

B.2 第2问第3问程序

Q2andQ3.py

```

#本程序及代码是在人工智能工具辅助下完成的, 使用Kimi模型
'''
模型版本: K2
开发机构: 月之暗面
颁布日期: 2025年7月11日
'''

import pandas as pd
import numpy as np
import networkx as nx
from collections import deque, defaultdict
import time, os, ast

# ===== 内存池管理类 =====
class CompactPool:
    def __init__(self, pool_type, capacity):
        self.type = pool_type
        self.capacity = capacity
        self.used_blocks = [] # (start, end, buf_id)

```

```

self.free_blocks = [(0, capacity)]
self.buf_to_offset = {}
self.spill_log = []
self.spill_cost = 0
self.total_allocated = 0
self.spill_counter = 0 # 用于生成新 spill 节点 ID

def alloc(self, buf_id, size, allow_spill=True, current_time
=0,
        free_times=None, buf_has_copy_in_dict=None,
        W1=1.0, W2=1.0, nodes=None, schedule=None):
    if size > self.capacity:
        return False, f"请求大小 {size} 超过内存池容量 {self.
            capacity}"

    # Best-Fit
    best_idx, best_size = -1, float('inf')
    for i, (start, end) in enumerate(self.free_blocks):
        block_size = end - start
        if block_size >= size and block_size < best_size:
            best_idx, best_size = i, block_size

    if best_idx != -1:
        start, end = self.free_blocks[best_idx]
        self.free_blocks.pop(best_idx)
        self.used_blocks.append((start, start + size, buf_id)
            )
        self.buf_to_offset[buf_id] = start
        self.total_allocated += size
        if start + size < end:
            self.free_blocks.append((start + size, end))
        self.free_blocks.sort()
        return True, "分配成功"

    # 如果失败, 尝试 spill (仅 UB/L1)
    if allow_spill and self.type in ['UB', 'L1']:
        best_pos, victim_bufs, total_cost = self.
            _find_best_spill_position(
                size, current_time, free_times or {},

```

```

        buf_has_copy_in_dict or {}, W1, W2
    )
    if victim_bufs:
        spill_success, _, _ = self.spill_victims(
            victim_bufs, current_time,
            free_times or {}, buf_has_copy_in_dict or {},
            W1, W2, nodes=nodes, schedule=schedule
        )
        if spill_success:
            return self.alloc(buf_id, size, allow_spill,
                               current_time, free_times,
                               buf_has_copy_in_dict, W1,
                               W2,
                               nodes=nodes, schedule=
                               schedule)

    return False, "分配失败"

def _find_best_spill_position(self, required_size,
                               current_time,
                               free_times,
                               buf_has_copy_in_dict,
                               W1=1.0, W2=1.0):
    candidate_positions = set()
    for start, _ in self.free_blocks:
        candidate_positions.add(start)
    for start, _, _ in self.used_blocks:
        candidate_positions.add(start)

    best_position, best_victims, best_total_cost = -1, [],
    float('inf')
    for pos in sorted(candidate_positions):
        if pos + required_size > self.capacity:
            continue
        overlapping_blocks, total_cost = [], 0
        for start, end, buf_id in self.used_blocks:
            if not (end <= pos or start >= pos +
                    required_size):
                size = end - start
                free_time = free_times.get(buf_id,

```

```

        current_time + 1000)
    tag = 1 if buf_has_copy_in_dict.get(buf_id,
        False) else 2
    remaining_time = max(1, free_time -
        current_time)
    cost = tag * W1 / size + W2 / remaining_time
    overlapping_blocks.append((buf_id, cost, size
        ))
    total_cost += cost
    if overlapping_blocks and total_cost <
        best_total_cost:
        best_position, best_victims, best_total_cost = (
            pos, [b for b, _, _ in overlapping_blocks],
            total_cost)
    return best_position, best_victims, best_total_cost

def spill_victims(self, victim_bufs, current_time,
    free_times, buf_has_copy_in_dict,
    W1=1.0, W2=1.0, nodes=None, schedule=None):
    new_spill_nodes = []
    for victim_buf in victim_bufs:
        victim_block = next((b for b in self.used_blocks if b
            [2] == victim_buf), None)
        if not victim_block:
            continue
        start, end, buf_id = victim_block
        size = end - start
        cost_coeff = 1 if buf_has_copy_in_dict.get(buf_id,
            False) else 2
        cost = cost_coeff * size
        self.spill_cost += cost
        self.spill_log.append((victim_buf, cost, current_time
            ))

    # === 显式生成新 spill 节点 ===
    if nodes is not None and schedule is not None:
        new_id = f"spill_{self.type}_{self.spill_counter}"
        "
        self.spill_counter += 1

```

```

        nodes.loc[new_id] = {
            "Op": "SPILL",
            "Type": self.type,
            "BufId": buf_id,
            "Size": size,
            "Cost": cost,
            "Cycles": 1,
            "Pipe": "DMA"
        }
        schedule.append(new_id)
        new_spill_nodes.append(new_id)

    self.free(victim_buf)
    return True, self.spill_cost, new_spill_nodes

def free(self, buf_id):
    victim_block = next((b for b in self.used_blocks if b[2]
        == buf_id), None)
    if not victim_block:
        return
    self.used_blocks.remove(victim_block)
    start, end, _ = victim_block
    self.total_allocated -= (end - start)
    self.free_blocks.append((start, end))
    self.free_blocks.sort()
    merged = []
    if self.free_blocks:
        s, e = self.free_blocks[0]
        for i in range(1, len(self.free_blocks)):
            ns, ne = self.free_blocks[i]
            if ns <= e:
                e = max(e, ne)
            else:
                merged.append((s, e))
                s, e = ns, ne
        merged.append((s, e))
    self.free_blocks = merged

def get_offset(self, buf_id):

```

```

        return self.buf_to_offset.get(buf_id, -1)

# ===== 多池管理器 =====
class MultiPoolManager:
    def __init__(self, capacities, buf_has_copy_in_dict):
        self.pools = {t: CompactPool(t, capacities[t]) for t in
            capacities}
        self.buf_has_copy_in_dict = buf_has_copy_in_dict
        self.global_spill_log, self.total_spill_cost = [], 0
        self.l0_alloc_failed = False

    def alloc(self, buf_id, size, buf_type, current_time,
        free_time, W1=1.0, W2=1.0, nodes=None, schedule=
            None):
        pool = self.pools[buf_type]
        allow_spill = buf_type not in ['L0A', 'L0B', 'L0C']
        success, message = pool.alloc(
            buf_id, size, allow_spill,
            current_time, {buf_id: free_time},
            self.buf_has_copy_in_dict, W1, W2,
            nodes=nodes, schedule=schedule
        )
        if not success and not allow_spill:
            self.l0_alloc_failed = True
        return success, message

    def free(self, buf_id, buf_type):
        self.pools[buf_type].free(buf_id)

    def get_offset(self, buf_id, buf_type):
        return self.pools[buf_type].get_offset(buf_id)

    def collect_spill_info(self):
        self.global_spill_log, self.total_spill_cost = [], 0
        for p in self.pools.values():
            self.global_spill_log.extend(p.spill_log)
            self.total_spill_cost += p.spill_cost
        return self.total_spill_cost, self.global_spill_log

```

```

# ===== 时钟数赋值 =====
def assign_cycles(nodes):
    def cycle_rule(r):
        op=str(r['Op']).upper()
        if op in ['ALLOC', 'FREE', 'COPY_IN', 'COPY_OUT']: return 0
        if 'Cycles' in r and pd.isna(r['Cycles']): return int(r[
            'Cycles'])
        if op in ['MTE1', 'MTE2', 'MTE3', 'MMAD', 'CUBE']:
            return max(1, int(r.get('Size', 1))//16)
        return 1
    nodes['Cycles']=nodes.apply(cycle_rule, axis=1)
    return nodes

# ===== 图构建 =====
def build_dag_fast(edges):
    G=nx.DiGraph()
    G.add_edges_from([(int(r.StartNodeId), int(r.EndNodeId)) for _,
        r in edges.iterrows()])
    return G

# ===== 贪心拓扑排序 (L0) =====
def greedy_topo_with_L0_pool(nodes, graph, capacities):
    node_attrs={nid:row.to_dict() for nid,row in nodes.iterrows()}
    indeg={n:graph.in_degree(n) for n in graph.nodes}
    ready=deque([n for n in graph.nodes if indeg[n]==0])
    order=[]; l0_pools={t:CompactPool(t, capacities[t]) for t in [
        'LOA', 'LOB', 'LOC']}
    while ready:
        min_cost, sel=float('inf'), None
        for n in list(ready):
            a=node_attrs[n]
            if a['Op']=="ALLOC" and a['Type'] in l0_pools:
                ok, _=l0_pools[a['Type']].alloc(a['BufId'], a['Size'],
                    allow_spill=False)
                if ok: l0_pools[a['Type']].free(a['BufId'])
                else: continue
            if a['Cost']<min_cost: min_cost, sel=a['Cost'], n
        if sel is None: sel=ready[0]

```

```

cur=sel; a=node_attrs[cur]
if a['Op']=="ALLOC" and a['Type'] in l0_pools:
    l0_pools[a['Type']].alloc(a['BufId'],a['Size'],
        allow_spill=False)
elif a['Op']=="FREE" and a['Type'] in l0_pools:
    l0_pools[a['Type']].free(a['BufId'])
for succ in graph.successors(cur):
    indeg[succ]-=1
    if indeg[succ]==0: ready.append(succ)
order.append(cur)
if cur in ready: ready.remove(cur)
return order

# ===== 时钟计算 =====
def compute_clock(schedule,nodes,edges):
    pred_end,pipe_last=defaultdict(int),defaultdict(int)
    S,E={},{}; edge_map=defaultdict(list)
    for _,r in edges.iterrows(): edge_map[int(r.StartNodeId)].
        append(int(r.EndNodeId))
    for nid in schedule:
        row=nodes.loc[nid]; p=row.Pipe; cycles=int(row['Cycles'])
        early=max(pred_end[nid],pipe_last[p]); S[nid]=early; E[
            nid]=early+cycles; pipe_last[p]=E[nid]
        for succ in edge_map[nid]: pred_end[succ]=max(pred_end[
            succ],E[nid])
    return S,E,max(E.values(),default=0)

def conservative_left_slide(schedule,nodes,edges):
    pred_end=defaultdict(int)
    for _,r in edges.iterrows(): pred_end[int(r.EndNodeId)]=max(
        pred_end[int(r.EndNodeId)],0)
    addr_free={}
    for i,nid in enumerate(schedule):
        row=nodes.loc[nid]
        if row.Op=='FREE': addr_free[row.BufId]=i
    pipe_last=defaultdict(int); S_new,E_new={},{}
    for i,nid in enumerate(schedule):
        row=nodes.loc[nid]; p=row.Pipe; cyc=int(row.Cycles)
        early=max(pred_end[nid],pipe_last[p],addr_free.get(row.

```

```

        get('BufId'), 0))
    S_new[nid]=early; E_new[nid]=early+cyc; pipe_last[p]=
        E_new[nid]
    return S_new, E_new, max(E_new.values(), default=0)

# ===== 数据读取 =====
def read_csv_optimized(case):
    nodes=pd.read_csv(f"{case}_Nodes.csv"); edges=pd.read_csv(f"{
        case}_Edges.csv")
    if 'Type' not in nodes: nodes['Type']=''
    if 'BufId' not in nodes: nodes['BufId']=-1
    if 'Size' not in nodes: nodes['Size']=0
    def cost(r):
        op=str(r['Op']).upper()
        if op=='ALLOC': return int(r['Size'])
        elif op=='FREE': return -int(r['Size'])
        else: return 0
    nodes['Cost']=nodes.apply(cost, axis=1)
    if "NodeId" in nodes: nodes.set_index("NodeId", inplace=True)
    elif "Id" in nodes: nodes.set_index("Id", inplace=True)
    nodes=assign_cycles(nodes); return nodes, edges

# ===== 输出 =====
def write_outputs(case, schedule, memory_alloc, spill_log,
    total_cost, folder):
    os.makedirs(folder, exist_ok=True); prefix=f"{folder}/{case}"
    with open(f"{prefix}_schedule.txt", "w") as f:
        for nid in schedule: f.write(f"{nid}\n")
    with open(f"{prefix}_memory.txt", "w") as f:
        for buf, off in memory_alloc.items(): f.write(f"{buf}:{off}
            }\n")
    with open(f"{prefix}_spill.txt", "w") as f:
        for rec in spill_log:
            if isinstance(rec, dict):
                if 'buf_id' in rec and 'new_offset' in rec: f.write
                    (f"{rec['buf_id']}:{rec['new_offset']}\n")
            elif isinstance(rec, (list, tuple)) and len(rec)>=2: f.
                write(f"{rec[0]}:{rec[1]}\n")

```

```

# ===== 主函数 =====
def main():
    capacities={'L1':4096, 'UB':1024, 'L0A':256, 'L0B':256, 'LOC':512}
    cases=["FlashAttention_Case0", "FlashAttention_Case1", "Matmul_Case0", "Matmul_Case1", "Conv_Case0", "Conv_Case1"]
    print("==== 二问+三问 ====")
    for case in cases:
        print(f"\n=== {case} ===")
        nodes, edges=read_csv_optimized(case); G=build_dag_fast(edges)
        schedule=greedy_topo_with_L0_pool(nodes, G, capacities)
        # 内存管理
        buf_free={}; [buf_free.setdefault(nodes.loc[nid].BufId, i)
            for i, nid in enumerate(schedule) if nodes.loc[nid].Op=='FREE']
        manager=MultiPoolManager(capacities, {})
        mem_alloc={}
        for i, nid in enumerate(schedule):
            row=nodes.loc[nid]
            if row.Op=='ALLOC':
                free_t=buf_free.get(row.BufId, i+1000)
                ok, _=manager.alloc(row.BufId, row.Size, row.Type, i, free_t,
                    nodes=nodes, schedule=schedule) # 传入 schedule
            if ok:
                off=manager.get_offset(row.BufId, row.Type)
                if off!=-1: mem_alloc[row.BufId]=off
            elif row.Op=='FREE': manager.free(row.BufId, row.Type)
        total_cost, spill_log=manager.collect_spill_info()
        # 写 Problem2
        write_outputs(case, schedule, mem_alloc, spill_log, total_cost, "Problem2")
        # 时钟计算
        S, E, T=compute_clock(schedule, nodes, edges)
        S_new, E_new, T_new=conservative_left_slide(schedule, nodes, edges)
        # 写 Problem3 (schedule 同二问, memory 同二问, 但 spill

```

```

        文件可不同)
    write_outputs(case, schedule, mem_alloc, spill_log,
                  total_cost, "Problem3")
    print(f"搬运量={total_cost}, 原周期={T}, 左滑后周期={
          T_new}")

if __name__=="__main__": main()

```

B.3 灵敏度分析程序

w.py

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import networkx as nx
from collections import deque, defaultdict
import time
import ast
import os

# ===== 内存池管理类 =====
class CompactPool:
    def __init__(self, pool_type, capacity):
        self.type = pool_type
        self.capacity = capacity
        self.used_blocks = [] # (start, end, buf_id)
        self.free_blocks = [(0, capacity)]
        self.buf_to_offset = {}
        self.spill_log = []
        self.spill_cost = 0
        self.total_allocated = 0

    def alloc(self, buf_id, size, allow_spill=True, current_time
              =0,
              free_times=None, buf_has_copy_in_dict=None,
              W1=1.0, W2=1.0):
        if size > self.capacity:
            return False, f"请求大小 {size} 超过内存池容量 {self.

```

```

        capacity}"

# Best-Fit
best_idx, best_size = -1, float('inf')
for i, (start, end) in enumerate(self.free_blocks):
    block_size = end - start
    if block_size >= size and block_size < best_size:
        best_idx, best_size = i, block_size

if best_idx != -1:
    start, end = self.free_blocks[best_idx]
    self.free_blocks.pop(best_idx)
    self.used_blocks.append((start, start + size, buf_id)
    )
    self.buf_to_offset[buf_id] = start
    self.total_allocated += size
    if start + size < end:
        self.free_blocks.append((start + size, end))
    self.free_blocks.sort()
    return True, "分配成功"

# 如果失败, 尝试 spill (仅 UB/L1)
if allow_spill and self.type in ['UB', 'L1']:
    best_pos, victim_bufs, total_cost = self._find_best_spill_position(
        size, current_time, free_times or {},
        buf_has_copy_in_dict or {}, W1, W2
    )
    if victim_bufs:
        spill_success, _ = self.spill_victims(
            victim_bufs, current_time,
            free_times or {}, buf_has_copy_in_dict or {},
            W1, W2
        )
        if spill_success:
            return self.alloc(buf_id, size, allow_spill,
                               current_time, free_times,
                               buf_has_copy_in_dict, W1,
                               W2)

```

```

    return False, "分配失败"

def _find_best_spill_position(self, required_size,
    current_time,
                                free_times,
                                buf_has_copy_in_dict,
                                W1=1.0, W2=1.0):
    candidate_positions = set()
    for start, _ in self.free_blocks:
        candidate_positions.add(start)
    for start, _, _ in self.used_blocks:
        candidate_positions.add(start)

    best_position, best_victims, best_total_cost = -1, [],
    float('inf')
    for pos in sorted(candidate_positions):
        if pos + required_size > self.capacity:
            continue
        overlapping_blocks, total_cost = [], 0
        for start, end, buf_id in self.used_blocks:
            if not (end <= pos or start >= pos +
                required_size):
                size = end - start
                free_time = free_times.get(buf_id,
                    current_time + 1000)
                tag = 1 if buf_has_copy_in_dict.get(buf_id,
                    False) else 2
                remaining_time = max(1, free_time -
                    current_time)
                cost = tag * W1 / size + W2 / remaining_time
                overlapping_blocks.append((buf_id, cost, size
                    ))
                total_cost += cost
        if overlapping_blocks and total_cost <
            best_total_cost:
            best_position, best_victims, best_total_cost = (
                pos, [b for b, _, _ in overlapping_blocks],
                total_cost)

```

```

    return best_position, best_victims, best_total_cost

def spill_victims(self, victim_bufs, current_time,
                  free_times, buf_has_copy_in_dict,
                  W1=1.0, W2=1.0):
    for victim_buf in victim_bufs:
        victim_block = next((b for b in self.used_blocks if b
                              [2] == victim_buf), None)
        if not victim_block:
            continue
        start, end, buf_id = victim_block
        size = end - start
        cost_coeff = 1 if buf_has_copy_in_dict.get(buf_id,
                                                    False) else 2
        cost = cost_coeff * size
        self.spill_cost += cost
        self.spill_log.append((victim_buf, cost, current_time
                               ))
        self.free(victim_buf)
    return True, self.spill_cost

def free(self, buf_id):
    victim_block = next((b for b in self.used_blocks if b[2]
                        == buf_id), None)
    if not victim_block:
        return
    self.used_blocks.remove(victim_block)
    start, end, _ = victim_block
    self.total_allocated -= (end - start)
    self.free_blocks.append((start, end))
    self.free_blocks.sort()
    merged = []
    if self.free_blocks:
        s, e = self.free_blocks[0]
        for i in range(1, len(self.free_blocks)):
            ns, ne = self.free_blocks[i]
            if ns <= e:
                e = max(e, ne)
            else:

```

```

        merged.append((s, e))
        s, e = ns, ne
        merged.append((s, e))
    self.free_blocks = merged

def get_offset(self, buf_id):
    return self.buf_to_offset.get(buf_id, -1)

# ===== 多池管理器 =====
class MultiPoolManager:
    def __init__(self, capacities, buf_has_copy_in_dict):
        self.pools = {t: CompactPool(t, capacities[t]) for t in
            capacities}
        self.buf_has_copy_in_dict = buf_has_copy_in_dict
        self.global_spill_log, self.total_spill_cost = [], 0
        self.l0_alloc_failed = False

    def alloc(self, buf_id, size, buf_type, current_time,
        free_time, W1=1.0, W2=1.0):
        pool = self.pools[buf_type]
        allow_spill = buf_type not in ['L0A', 'L0B', 'L0C']
        success, message = pool.alloc(buf_id, size, allow_spill,
            current_time, {buf_id:
                free_time},
            self.buf_has_copy_in_dict,
            W1, W2)

        if not success and not allow_spill:
            self.l0_alloc_failed = True
        return success, message

    def free(self, buf_id, buf_type):
        self.pools[buf_type].free(buf_id)

    def get_offset(self, buf_id, buf_type):
        return self.pools[buf_type].get_offset(buf_id)

    def collect_spill_info(self):
        self.global_spill_log, self.total_spill_cost = [], 0
        for p in self.pools.values():

```

```

        self.global_spill_log.extend(p.spill_log)
        self.total_spill_cost += p.spill_cost
    return self.total_spill_cost, self.global_spill_log

# ===== 时钟数赋值 =====
def assign_cycles(nodes):
    def cycle_rule(row):
        op = str(row['Op']).upper()
        if op in ['ALLOC', 'FREE', 'COPY_IN', 'COPY_OUT']:
            return 0
        if 'Cycles' in row and pd.notna(row['Cycles']):
            return int(row['Cycles'])
        if op in ['MTE1', 'MTE2', 'MTE3', 'MMAD', 'CUBE']:
            return max(1, int(row.get('Size', 1)) // 16)
        return 1
    nodes['Cycles'] = nodes.apply(cycle_rule, axis=1)
    return nodes

# ===== 拓扑排序 =====
def build_dag_fast(edges):
    G = nx.DiGraph()
    G.add_edges_from([(int(r.StartNodeId), int(r.EndNodeId)) for
        _, r in edges.iterrows()])
    return G

def greedy_topo_with_L0_pool(nodes, graph, capacities):
    node_attrs = {nid: row.to_dict() for nid, row in nodes.
        iterrows()}
    in_degree = {n: graph.in_degree(n) for n in graph.nodes}
    zero_degree_nodes = deque([n for n in graph.nodes if
        in_degree[n] == 0])
    execution_order = []

    l0_pools = {t: CompactPool(t, capacities[t]) for t in ['L0A',
        'L0B', 'L0C']}

    while zero_degree_nodes:
        min_cost, selected_node = float('inf'), None
        for node in list(zero_degree_nodes):

```

```

        attr = node_attrs[node]
        if attr['Op'] == "ALLOC" and attr['Type'] in l0_pools:
            :
            pool = l0_pools[attr['Type']]
            success, _ = pool.alloc(attr['BufId'], attr['Size'], allow_spill=False)
            if success:
                pool.free(attr['BufId'])
            else:
                continue
        if attr['Cost'] < min_cost:
            min_cost, selected_node = attr['Cost'], node
    if selected_node is None:
        selected_node = zero_degree_nodes[0]
    current_node = selected_node
    current_attr = node_attrs[current_node]
    if current_attr['Op'] == "ALLOC" and current_attr['Type'] in l0_pools:
        l0_pools[current_attr['Type']].alloc(current_attr['BufId'], current_attr['Size'], allow_spill=False)
    elif current_attr['Op'] == "FREE" and current_attr['Type'] in l0_pools:
        l0_pools[current_attr['Type']].free(current_attr['BufId'])
    for succ in graph.successors(current_node):
        in_degree[succ] -= 1
        if in_degree[succ] == 0:
            zero_degree_nodes.append(succ)
    execution_order.append(current_node)
    if current_node in zero_degree_nodes:
        zero_degree_nodes.remove(current_node)
    return execution_order

```

===== 时钟调度计算 =====

```

def compute_clock(schedule, nodes, edges):
    pred_end, pipe_last = defaultdict(int), defaultdict(int)
    S, E = {}, {}
    edge_map = defaultdict(list)
    for _, r in edges.iterrows():

```

```

        edge_map[int(r.StartNodeId)].append(int(r.EndNodeId))
    for nid in schedule:
        row = nodes.loc[nid]
        p = row.Pipe
        cycles = int(row['Cycles'])
        early = max(pred_end[nid], pipe_last[p])
        S[nid], E[nid] = early, early + cycles
        pipe_last[p] = E[nid]
        for succ in edge_map[nid]:
            pred_end[succ] = max(pred_end[succ], E[nid])
    return S, E, max(E.values(), default=0)

# ===== 数据读取 =====
'''
def read_csv_optimized(case):
    nodes = pd.read_csv(f"{case}_Nodes.csv")
    edges = pd.read_csv(f"{case}_Edges.csv")
    if 'Type' not in nodes: nodes['Type'] = ''
    if 'BufId' not in nodes: nodes['BufId'] = -1
    if 'Size' not in nodes: nodes['Size'] = 0
    nodes['Cost'] = nodes.apply(lambda r: r['Size'] if r['Op'] ==
                                'ALLOC' else -r['Size'] if r['Op'] == 'FREE' else 0, axis
                                =1)
    if "NodeId" in nodes: nodes.set_index("NodeId", inplace=True)
    elif "Id" in nodes: nodes.set_index("Id", inplace=True)
    return assign_cycles(nodes), edges
'''

def read_csv_optimized(case):
    nodes = pd.read_csv(f"{case}_Nodes.csv")
    edges = pd.read_csv(f"{case}_Edges.csv")

    if 'Type' not in nodes:
        nodes['Type'] = ''
    if 'BufId' not in nodes:
        nodes['BufId'] = -1
    if 'Size' not in nodes:
        nodes['Size'] = 0

```

```

def compute_cost(r):
    if str(r['Op']).upper() == 'ALLOC':
        return int(r['Size'])
    elif str(r['Op']).upper() == 'FREE':
        return -int(r['Size'])
    else:
        return 0

nodes['Cost'] = nodes.apply(compute_cost, axis=1)

if "NodeId" in nodes:
    nodes.set_index("NodeId", inplace=True)
elif "Id" in nodes:
    nodes.set_index("Id", inplace=True)

# 加上时钟数赋值
nodes = assign_cycles(nodes)
return nodes, edges

def parameter_scan(cases, capacities, W1_list, W2_list,
out_dir_scan="ParamScan"):
    os.makedirs(out_dir_scan, exist_ok=True)

    for case in cases:
        print(f"\n=== 参数扫描 {case} ===")
        nodes, edges = read_csv_optimized(case)
        G = build_dag_fast(edges)
        schedule = greedy_topo_with_L0_pool(nodes, G, capacities)

        buf_free_positions = {}
        for i, nid in enumerate(schedule):
            row = nodes.loc[nid]
            if row.Op == 'FREE':
                buf_free_positions[row.BufId] = i

        # 结果矩阵
        spill_matrix = np.zeros((len(W1_list), len(W2_list)))

```

```

time_matrix = np.zeros((len(W1_list), len(W2_list)))

for i, w1 in enumerate(W1_list):
    for j, w2 in enumerate(W2_list):
        buf_has_copy_in_dict = {}
        manager = MultiPoolManager(capacities,
                                    buf_has_copy_in_dict)
        for k, nid in enumerate(schedule):
            row = nodes.loc[nid]
            if row.Op == 'ALLOC':
                free_time = buf_free_positions.get(row.
                                                    BufId, k + 1000)
                manager.alloc(row.BufId, row.Size, row.
                              Type, k, free_time, W1=w1, W2=w2)
            elif row.Op == 'FREE':
                manager.free(row.BufId, row.Type)

        total_cost, _ = manager.collect_spill_info()
        _, _, T_total = compute_clock(schedule, nodes,
                                       edges)

        spill_matrix[i, j] = total_cost
        time_matrix[i, j] = T_total

# === 绘制热力图 ===
W1_grid, W2_grid = np.meshgrid(W2_list, W1_list)

# === 绘制热力图（带网格） ===
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Spill Cost
im1 = axes[0].imshow(spill_matrix, cmap="viridis",
                     extent=[min(W2_list), max(W2_list),
                              min(W1_list), max(W1_list)],
                     origin="lower", aspect="auto")
axes[0].set_title(f"{case} - Spill Cost")
axes[0].set_xlabel("W2")
axes[0].set_ylabel("W1")
axes[0].set_xticks(W2_list)

```

```

axes[0].set_yticks(W1_list)
axes[0].grid(color='white', linestyle='--', linewidth
            =0.5)
plt.colorbar(im1, ax=axes[0])

# Execution Time
im2 = axes[1].imshow(time_matrix, cmap="plasma",
                    extent=[min(W2_list), max(W2_list),
                            min(W1_list), max(W1_list)],
                    origin="lower", aspect="auto")
axes[1].set_title(f"{case} - Execution Time")
axes[1].set_xlabel("W2")
axes[1].set_ylabel("W1")
axes[1].set_xticks(W2_list)
axes[1].set_yticks(W1_list)
axes[1].grid(color='white', linestyle='--', linewidth
            =0.5)
plt.colorbar(im2, ax=axes[1])

plt.tight_layout()
plt.savefig(os.path.join(out_dir_scan, f"{case}_heatmap.
                        png"))
plt.close()

# fig, axes = plt.subplots(1, 2, figsize=(12, 5))
# im1 = axes[0].imshow(spill_matrix, cmap="viridis",
#                     extent=[min(W2_list), max(W2_list)
#                             , min(W1_list), max(W1_list)],
#                     origin="lower", aspect="auto")
# axes[0].set_title(f"{case} - Spill Cost")
# axes[0].set_xlabel("W2")
# axes[0].set_ylabel("W1")
# plt.colorbar(im1, ax=axes[0])

# im2 = axes[1].imshow(time_matrix, cmap="plasma",
#                     extent=[min(W2_list), max(W2_list)
#                             , min(W1_list), max(W1_list)],
#                     origin="lower", aspect="auto")

```

```

# axes[1].set_title(f"{case} - Execution Time")
# axes[1].set_xlabel("W2")
# axes[1].set_ylabel("W1")
# plt.colorbar(im2, ax=axes[1])

# plt.tight_layout()
# plt.savefig(os.path.join(out_dir_scan, f"{case}_heatmap.png"))
# plt.close()

print(f"完成 {case} 参数扫描图, 已保存到 {out_dir_scan}/{case}_heatmap.png")

def conservative_left_slide(final_schedule, nodes, edges):
    """
    第三问: 零新增 SPILL 前提下的保守时间左滑
    只把节点压到理论最早可启动时刻, 不改变顺序、不新增 SPILL、不调地址
    返回: 新起止时钟 S_new, E_new, 新总周期 T_new
    """
    from collections import defaultdict
    import pandas as pd

    # 1. 前驱最晚结束表 (图依赖)
    pred_end = defaultdict(int)
    for _, r in edges.iterrows():
        u, v = int(r.StartNodeId), int(r.EndNodeId)
        pred_end[v] = max(pred_end[v], 0) # 后面更新为 E[u]

    # 2. 地址空出表 (FREE 序列位置当时刻)
    addr_free = {} # bufId -> FREE 时刻
    for i, nid in enumerate(final_schedule):
        row = nodes.loc[nid]
        if row.Op == 'FREE':
            addr_free[row.BufId] = i

    # 3. 同列串行尾巴 & 逐节点左滑
    pipe_last = defaultdict(int) # Pipe -> 列尾时间

```

```

S_new, E_new = {}, {}

for i, nid in enumerate(final_schedule):
    row = nodes.loc[nid]
    p = row.Pipe
    cycles = int(row.Cycles) if pd.notna(row.get('Cycles',
        None)) else 0

    # 理论最早可启动 = 图依赖 vs 列尾 vs 地址空出
    early = max(pred_end[nid], pipe_last[p], addr_free.get(
        row.get('BufId'), 0))
    S_new[nid] = early
    E_new[nid] = early + cycles
    pipe_last[p] = E_new[nid]

T_new = max(E_new.values(), default=0)
return S_new, E_new, T_new

def main():
    capacities = {'L1': 4096, 'UB': 1024, 'LOA': 256, 'LOB': 256,
        'LOC': 512}
    cases = ["FlashAttention_Case0", "FlashAttention_Case1",
        "Matmul_Case0", "Matmul_Case1", "Conv_Case0", "
        Conv_Case1"]

    out_dir2 = "Problem2_Output"
    out_dir3 = "Problem3_Output"
    os.makedirs(out_dir2, exist_ok=True)
    os.makedirs(out_dir3, exist_ok=True)

    print("==== 二问：数据搬运量 + 三问：流水时钟 ====")
    for case in cases:
        print(f"\n=== 处理 {case} ===")
        try:
            # === 读入数据 ===
            nodes, edges = read_csv_optimized(case)
            G = build_dag_fast(edges)
            schedule = greedy_topo_with_L0_pool(nodes, G,
                capacities)

```

```

# === 第二问：内存池管理模拟 ===
buf_free_positions = {}
for i, nid in enumerate(schedule):
    row = nodes.loc[nid]
    if row.Op == 'FREE':
        buf_free_positions[row.BufId] = i

buf_has_copy_in_dict = {}
manager = MultiPoolManager(capacities,
    buf_has_copy_in_dict)

for i, nid in enumerate(schedule):
    row = nodes.loc[nid]
    if row.Op == 'ALLOC':
        free_time = buf_free_positions.get(row.BufId,
            i + 1000)
        manager.alloc(row.BufId, row.Size, row.Type,
            i, free_time)
    elif row.Op == 'FREE':
        manager.free(row.BufId, row.Type)

total_cost, spill_log = manager.collect_spill_info()

# === 第三问：时钟调度 ===
S, E, T_total = compute_clock(schedule, nodes, edges)

slide_start = time.time()
S_new, E_new, T_new = conservative_left_slide(
    schedule, nodes, edges)
slide_time = time.time() - slide_start

print(f"搬运量: {total_cost}, 流水总时钟: {T_total}")
print(f"搬运量: {total_cost}, SPILL次数: {len(
    spill_log)}")
print(f"左滑耗时: {slide_time:.3f}s, 左滑后周期: {
    T_new}, 时间下降: {(T_total - T_new) / T_total *
    100:.2f}%")

```

```

# ===== 文件输出 =====
# 第二问输出
with open(os.path.join(out_dir2, f"{case}_schedule.
txt"), "w") as f:
    f.write("\n".join(str(n) for n in schedule))

with open(os.path.join(out_dir2, f"{case}_memory.txt"
), "w") as f:
    for pool in manager.pools.values():
        for buf, offset in pool.buf_to_offset.items():
            :
            f.write(f"{buf}:{offset}\n")

with open(os.path.join(out_dir2, f"{case}_spill.txt")
, "w") as f:
    for (buf, _, _) in spill_log:
        new_offset = (manager.get_offset(buf, 'UB')
or
manager.get_offset(buf, 'L1'))
        if new_offset != -1:
            f.write(f"{buf}:{new_offset}\n")

# 第三问输出
with open(os.path.join(out_dir3, f"{case}_schedule.
txt"), "w") as f:
    f.write("\n".join(str(n) for n in schedule))

with open(os.path.join(out_dir3, f"{case}_memory.txt"
), "w") as f:
    for pool in manager.pools.values():
        for buf, offset in pool.buf_to_offset.items():
            :
            f.write(f"{buf}:{offset}\n")

with open(os.path.join(out_dir3, f"{case}_spill.txt")
, "w") as f:
    for (buf, _, _) in spill_log:
        new_offset = (manager.get_offset(buf, 'UB')
or

```

```

manager.get_offset(buf, 'L1'))
    if new_offset != -1:
        f.write(f"{buf}:{new_offset}\n")

except Exception as e:
    print(f"处理 {case} 时出错: {e}")

if __name__ == "__main__":
    capacities = {'L1': 4096, 'UB': 1024, 'LOA': 256, 'LOB': 256,
                  'LOC': 512}
    cases = ["FlashAttention_Case0", "FlashAttention_Case1",
             "Matmul_Case0", "Matmul_Case1", "Conv_Case0", "
             Conv_Case1"]

    # 参数扫描范围 (可以改)
    W1_list = [0.5, 1.0, 2.0]
    W2_list = [0.5, 1.0, 2.0]

    parameter_scan(cases, capacities, W1_list, W2_list)

```